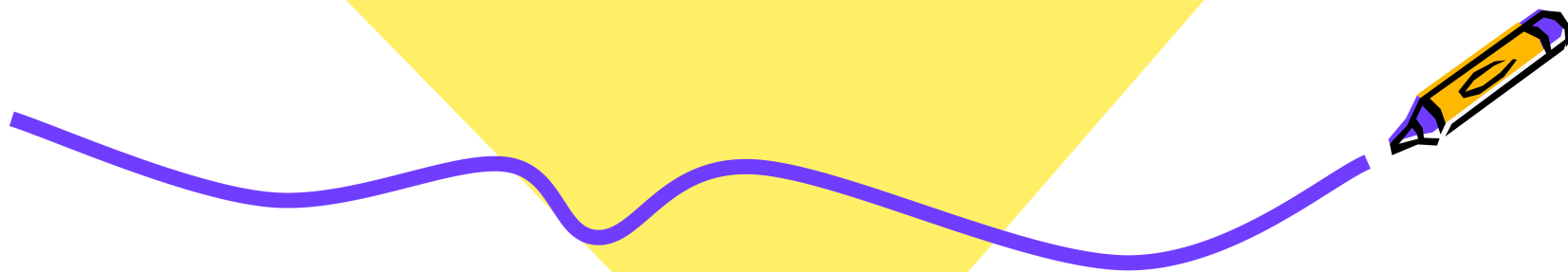




网络安全

哈尔滨工业大学计算机科学与技术学院
计算机网络与信息安全技术研究中心



考核要求



- 课堂笔记（手写）+平时 20%
- 程序报告30%
- 期末考试50%
- 每个部分不得低于该部分成绩的45%



网络空间

• 网络空间（Cyberspace）



网络空间四要素：网络空间载体（设施）；网络空间资源（数据）
网络活动主体（用户）；网络活动形式（操作）

网络空间载体：设施，信息通信技术系统的集合

网络操作对象：数据，表达人类所能理解的意图的信号状态

网络活动主体：用户，网络活动的主体要素，属于人的代理

网络活动形式：操作，对数据的加工、存储、传输、展示等服务形式

网络空间的一般性定义：网络空间是一种人造的电磁空间，其以互联网、各种通信系统与电信网、各种传播系统与广电网、各种计算机系统、各类关键工业设施中的嵌入式处理器和控制器等信息通信技术基础设施为**载体**，**用户**通过在其上对**数据**进行创造、存储、改变、传输、使用、展示等**操作**，以实现特定的信息通信技术活动。

在这个定义中，“载体”、“数据”是在技术层面反映出“Cyber”的属性；“用户”、“操作”是在社会层面反映出“Space”的属性。2



网络空间安全

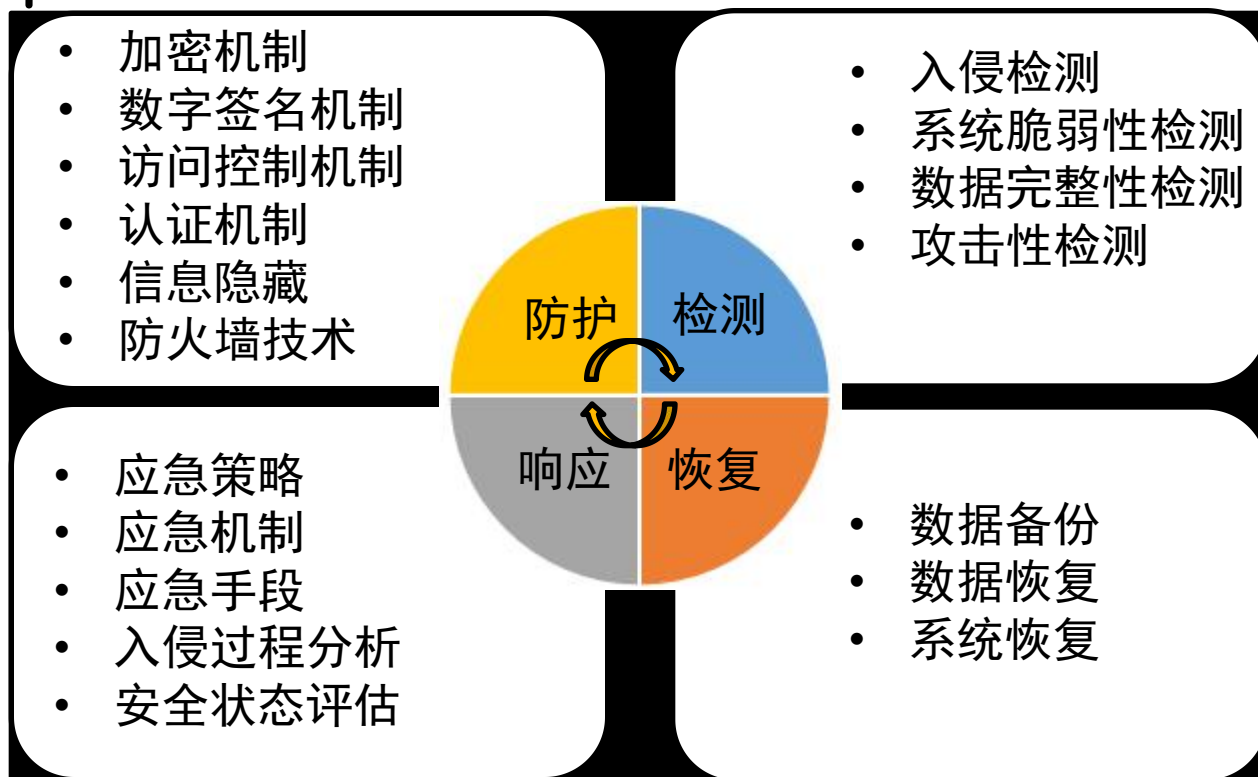
- 网络空间安全（Cyberspace Security）

- 方滨兴：在信息通信技术的硬件、代码、数据、应用4个层面，围绕着信息的获取、传输、处理、利用4个核心功能，针对网络空间的设施、数据、用户、操作4个核心要素来采取安全措施，以确保网络空间的机密性、可鉴别性、可用性、可控性4个核心安全属性得到保障，让信息通信技术系统能够提供安全、可信、可靠、可控的服务，面对网络空间攻防对抗的态势，通过信息、软件、系统、服务方面的确保手段、事先预防、事前发现、事中响应、事后恢复的应用措施，以及国家网络空间主权的行使，既要应对信息通信技术系统及其所受到的攻击，也要应对信息通信技术相关活动的衍生出政治安全、经济安全、文化安全、社会安全与国防安全的问题



PDRR模型

- DoD提出：防护（Protection）、检测（Detection）、恢复（Recovery）、响应（Response）



P2DR模型



- ISC提出



网络扫描技术



- 什么是网络扫描？

- 使用网络扫描软件对特定目标进行各种试探性通信，以获取目标信息的行为。

- 网络扫描的目的

- 识别目标主机的工作状态（开/关机）

主机扫描

- 识别目标主机端口的状态（监听/关闭）

端口扫描

- 识别目标主机的操作系统类型

操作系统识别

- 识别目标系统可能存在的漏洞

漏洞扫描

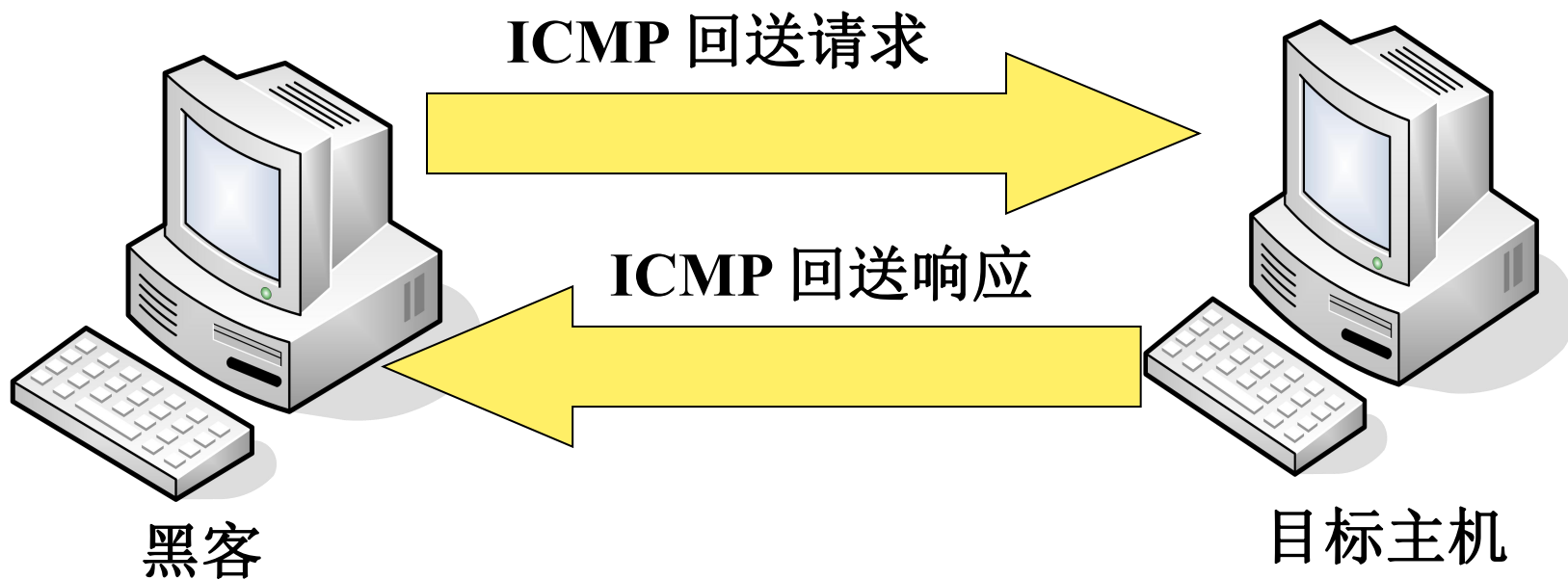


主机扫描

- 向目标主机发送探测数据包，根据是否收到响应来判断主机的工作状态。
- ICMP
 - ICMP Echo
 - ICMP Non-Echo
- IP
 - 异常的IP数据报首部
 - 错误的分片



ICMP Echo扫描(1/5)



结论：目标主机在运行



ICMP Echo扫描(2/5)



结论：目标主机未开机



ICMP Non-Echo扫描

- 利用其它类型的ICMP报文进行扫描

ICMP 报文种类	类型的值	ICMP报文的类型
差错报告报文	3	终点不可达
	4	源站抑制
	11	时间超过
	12	参数问题
	5	改变路由
询问报文	8或0	回送请求或回答
	13或14	时间戳请求或回答
	17或18	地址掩码请求或回答
	10或9	路由器询问或通告



异常的IP数据报首部：参数错

- 主机在收到首部异常（Header Length Field、IP Options Field、Version Number）的IP数据报时应当返回“参数问题”的ICMP报文。



异常的IP数据报首部：目标不可达



- 向目标主机发送的IP包中填充错误的字段值，目标主机或过滤设备会反馈ICMP Destination Unreachable信息。



错误的IP数据报分片

- 由于缺少分片而无法完成IP数据报重组（超时）时，主机应当回应“分片重组超时”的ICMP报文。



(一) TCP Connect扫描(1/2)

- 尝试同目标端口建立正常的**TCP**连接(直接调用系统提供的**connect(...)**函数)。
- 连接建立成功
 - 结论：目标端口开放
- 连接建立失败
 - 结论：目标端口关闭



TCP Connect扫描的特点(2/2)

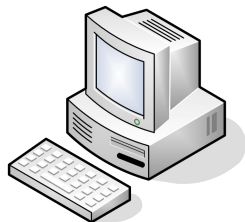


- 优点
 - 稳定可靠，不需要特殊的权限。
- 缺点
 - 扫描方式不隐蔽，服务器会记录下客户机的连接行为。
- 如何隐藏扫描行为？

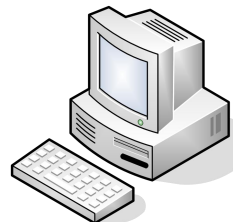


(二) SYN扫描(1/3)

主机 A



主机 B



SYN



目标端口

SYN, ACK



RST

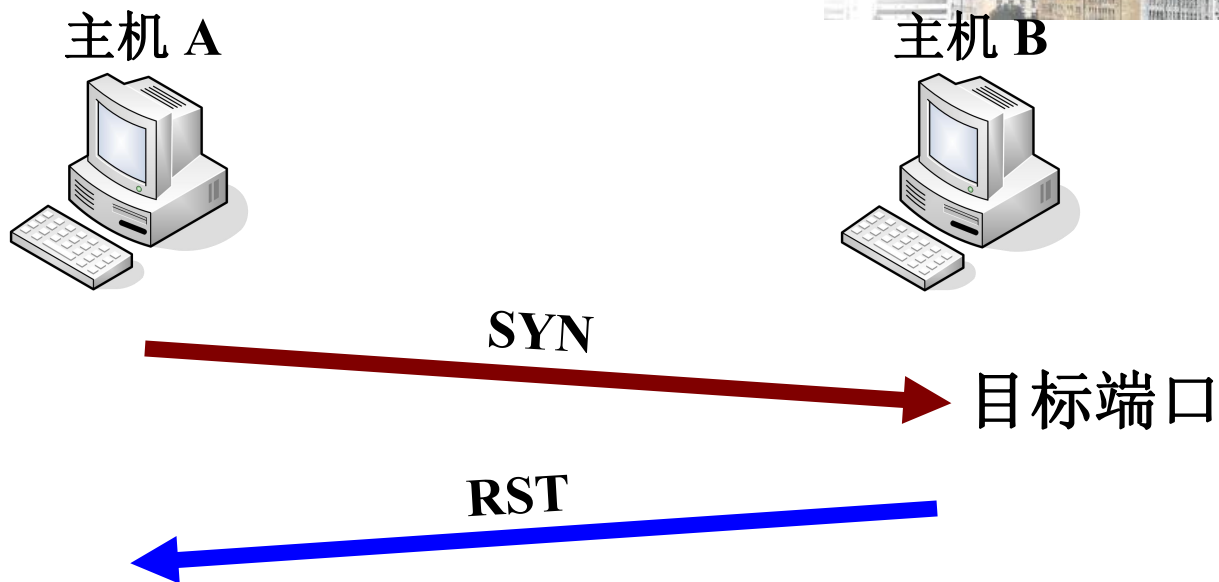


结论：端口开放



哈尔滨工业大学
HARBIN INSTITUTE OF TECHNOLOGY

SYN扫描(2/3)



结论：端口关闭



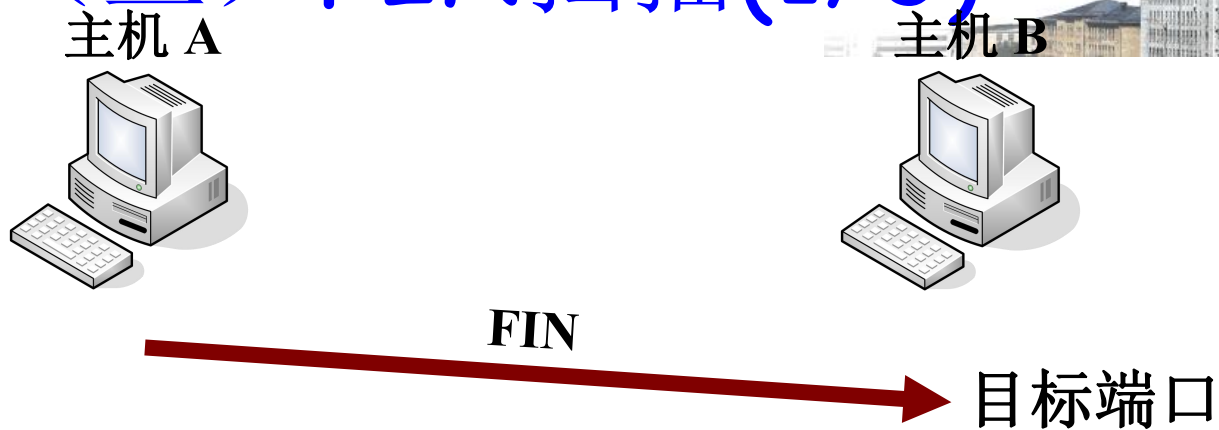
SYN扫描的特点(3/3)



- 优点
 - 很少有系统会记录这样的行为。
- 缺点
 - 需要管理员权限才可以构造这样的**SYN**数据包。



(三) FIN扫描(1/3)

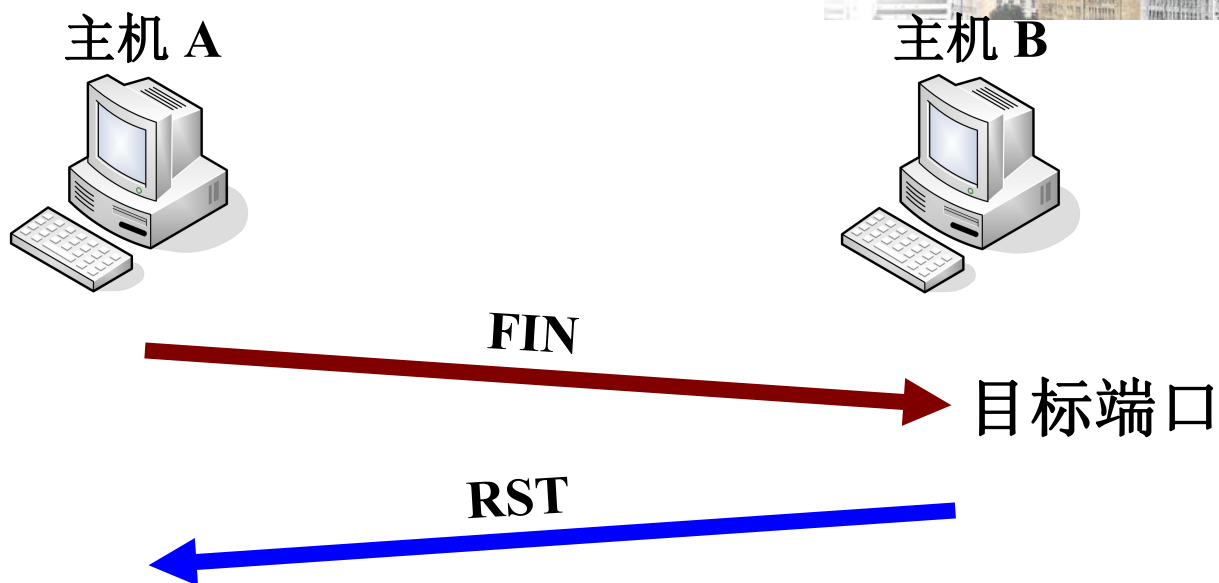


未收到任何响应

结论：端口开放



FIN扫描(2/3)



结论：端口关闭



FIN扫描的特点(3/3)

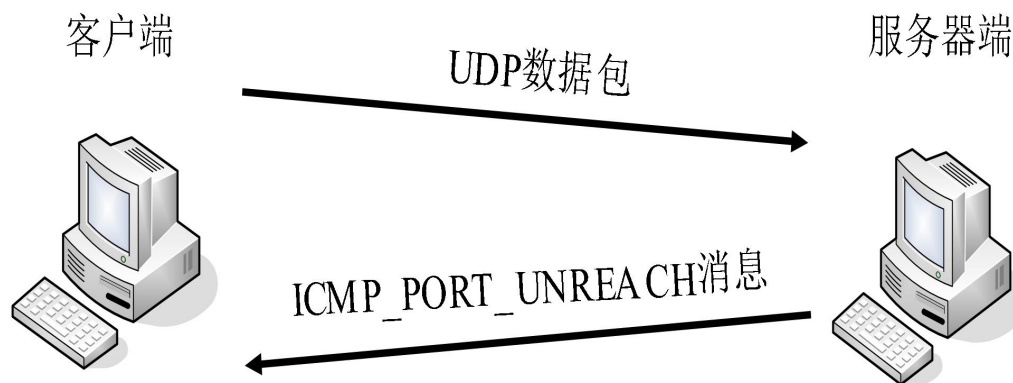


- 优点
 - 不是**TCP**建立连接的过程，比较隐蔽。
- 缺点
 - 与**SYN**扫描类似，也需要构造专门的数据包。
 - 只适用于**Unix**系统的目标主机，**Windows**系统总是发送**RST**报文段。



UDP扫描

- UDP没有连接建立过程，该如何判断一个UDP端口打开了呢？
- 依据：扫描主机向目标主机的UDP端口发送UDP数据包，如果目标端口处于监听状态，将不会做出任何响应；而如果目标端口处于关闭状态，将会返回ICMP_PORT_UNREACH错误。



IDS与扫描



- 对于**IDS**来说，几乎所有的**IDS**都能够准确地检测出扫描。
- 但是如果对同一主机不同端口进行分布式扫描，且间隔时间足够长，那么几乎没有什么可靠的办法可监测到这种扫描



• 安全性分析 IPv4

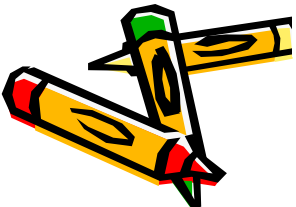
- IPv4协议没有认证机制：
 - 没有消息源认证：源地址假冒
 - 没有完整性认证：篡改
- IPv4没有加密机制
 - 无机密性：监听应用数据
 - 泄露拓扑等信息：网络侦察
- 无带宽控制：
 - DDoS攻击



IPv6



- 从IPv4向IPv6过渡采用逐步演进的方法，IETF推荐的过渡方案主要有：
 - 双协议栈(dual stack)
 - 隧道(tunneling)
 - 网络地址转换



IPv6 安全



- 安全问题

- IPv4向IPv6过渡技术的安全风险
- 无状态地址自动配置的安全风险
- IPv6中PKI管理系统的安全风险
- IPv6编址机制的隐患



IPv4向IPv6过渡技术的安全风险

- 黑客可以利用两种协议中存在的安全弱点以及漏洞进行协调攻击，或者利用协调不足来逃避检测
- 一种协议的漏洞可能会影响另一种协议的正常工作
- 隧道机制只是进行简单的封装，不进行任何的严格检查，会带来更复杂的安全问题



RIP协议安全

- **RIPv1**不支持认证，且使用不可靠的**UDP**协议作为传输协议，安全性较差。
- 如果在没有认证保护的情况下，攻击者可以轻易伪造**RIP** 路由更新信息，并向邻居路由器发送，伪造内容为目的网络地址、子网掩码地址与下一跳地址，经过若干轮的路由更新，网络通信将面临瘫痪的风险



RIP协议安全

- 对于不安全的**RIP**协议，中小型网络通常可采取的防范措施包括：
 - ①将路由器的某些接口配置为被动接口，配置为被动接口后，该接口停止向它所在的网络广播路由更新报文，但是允许它接收来自其他路由器的更新报文；
 - ②配置路由器的访问控制列表，只允许某些源**IP** 地址的路由更新报文进入列表



OSPF安全



- OSPF攻击方式

- 最大年龄（*Max Age attack*）攻击
- 序列号加1（*Sequence++*）攻击
- 最大序列号攻击
- 重放攻击
- 篡改攻击



最大年龄（Max Age attack）攻击



- **LSA**（链路状态通告）的最大年龄为**1h**
- 攻击者发送带有最大年龄设置的**LSA**信息报文
- 最开始的路由器通过产生刷新来发送这个**LSA**，而后引起**age**项中的突然改变值的竞争
- 攻击者持续的插入这个报文给整个路由群，将导致网络混乱和**DOS**



序列号加1 (Sequence++) 攻击

- OSPF根据LSA 的序号字段来判断是否是旧的LSA。
- 序列号越大LSA越新
- 攻击者持续插入较大LSA序列号报文时
- 最开始的路由器就会产生并发送自己更新的LSA序列号来与攻击者序列号竞争
- 导致网络不稳定和DOS



最大序列号攻击

- 根据OSPF协议的规定，当发送最大序列号（0X7FFFFFFFF）的网络设备再次发送报文前，要求其他设备也将序列号重置，OSPF停15min
- 攻击者插入一个最大序列号的LSA报文，将触发初始化过程
- 攻击者不断修改发出的lsa序列号，就会造成网络运行的不稳定



重放攻击



- 问候（**hello**）报文的重放攻击有两种方式：
 - 报文中列出了最近发现的路由器，所以攻击者重放**hello**报文给该报文的产生者，产生者不能再列表中查找到自己，认为该链路不是可双向通信的，此时将设置邻居的状态为Init状态，阻止建立邻接关系



重放攻击



- 问候（**hello**）报文的重放攻击有两种方式：
 - 更高序列号攻击，攻击者重放一个**hello**报文，该报文比原来报文具有更高的序列号，目的路由器将忽略真实报文，直到受到一个更高序列号的报文。如果在RouterDeadInterval内目的路由器没有受到一个**hello**，将不能维持邻居关系



重放攻击



- **LSA**报文的重放攻击：
 - 攻击者重放一个与拓扑不符的**LSA**，并洪泛出去，将序列号改为最新。
 - 各路由器接收后，触发最短路径计算
 - 源路由器收到后，将产生一个更高序列号的**LSA**并洪泛出去，又会触发**spf**计算
 - 频繁的计算会导致路由器性能下降



数字大炮



- 利用**BGP**路由正常工作中的路由表更新机制，在网络上制造某些通信链路的时断时续的震荡效应，是网络的路由器频繁的更新路由表，最终当震荡的路径数量越多，震荡的频率足够高的时候，网络上的路由器处于瘫痪状态
- 现有的**BGP**内置故障保护措施几乎无能为力

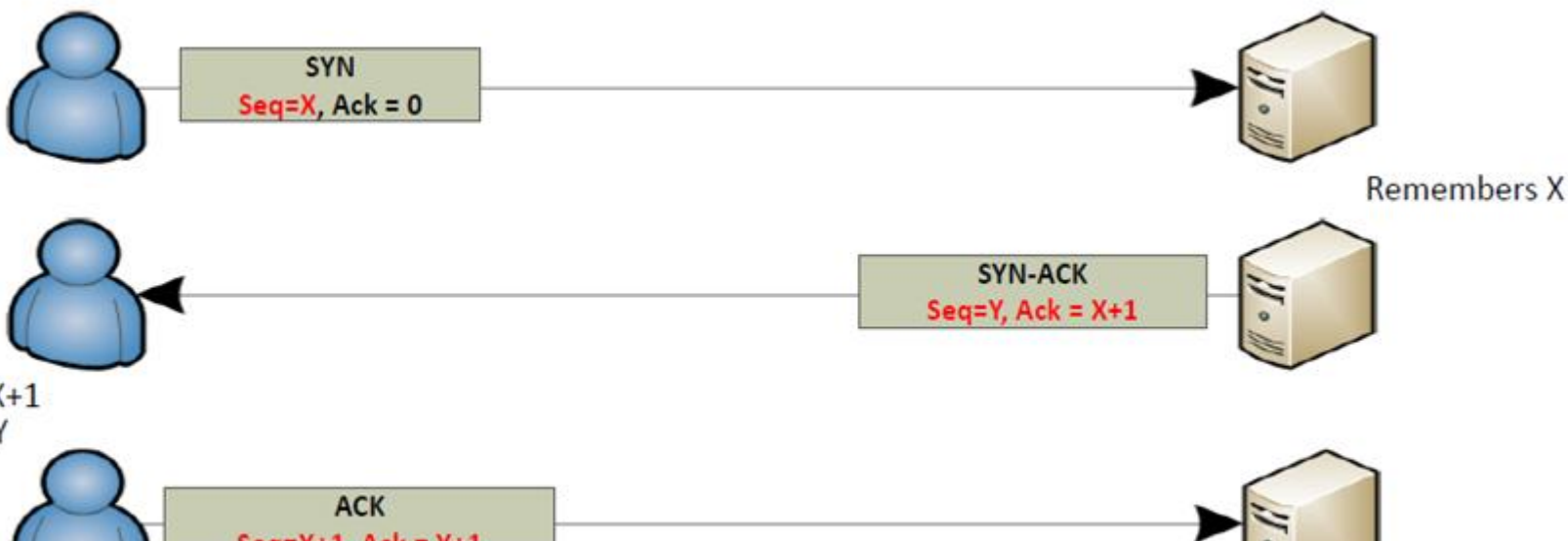


TCP协议

- 连接劫持:

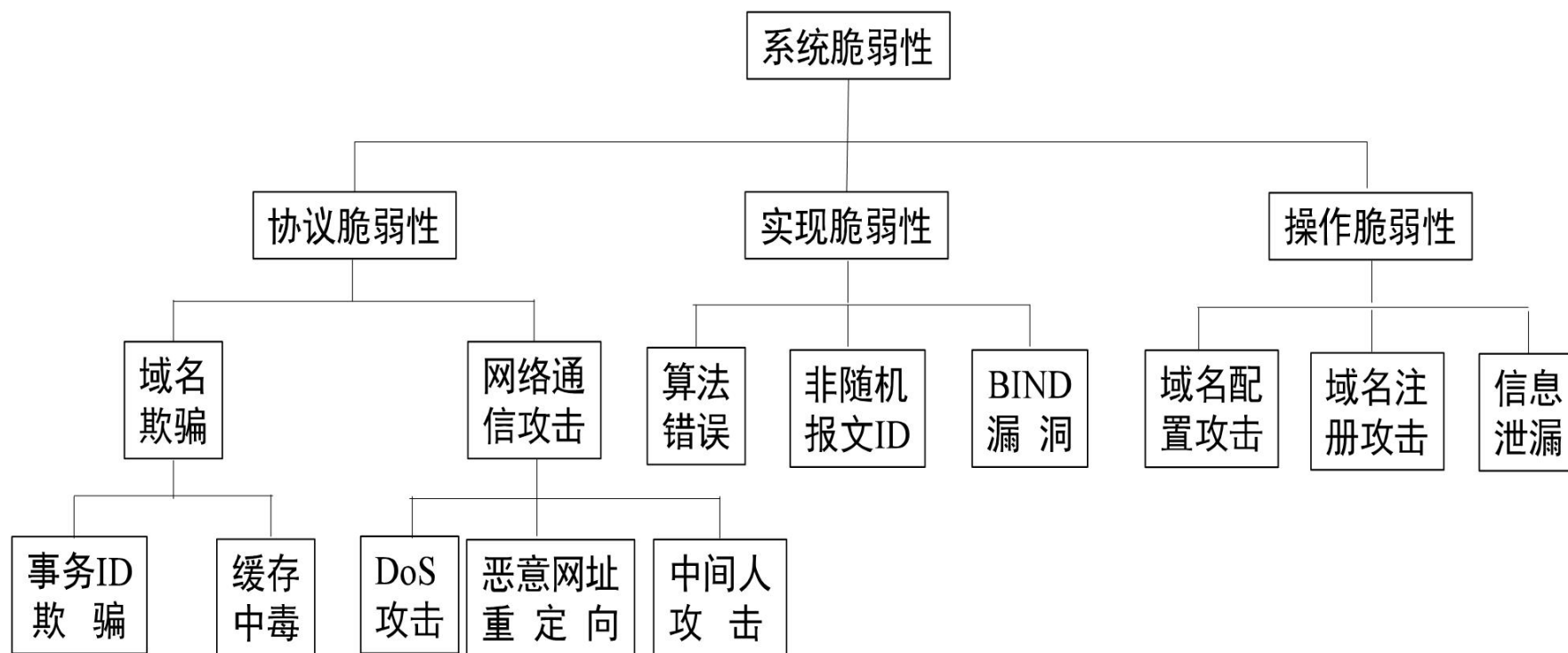
只要TCP包中的源端口、目的端口、Seq、Ack正确，即可被正确接收。当得到入侵者构造的TCP数据包，协议会假设数据包是来源于TCP连接中另一方的合法数据包，并且发送响应包到（入侵者构造的数据包中设置的IP地址）。随后，原来的TCP连接会由于计数器不匹配而断开连接。

关键：猜测Seq、Ack，如果是旁路劫持还需猜测源端口号



DNS

• 安全问题

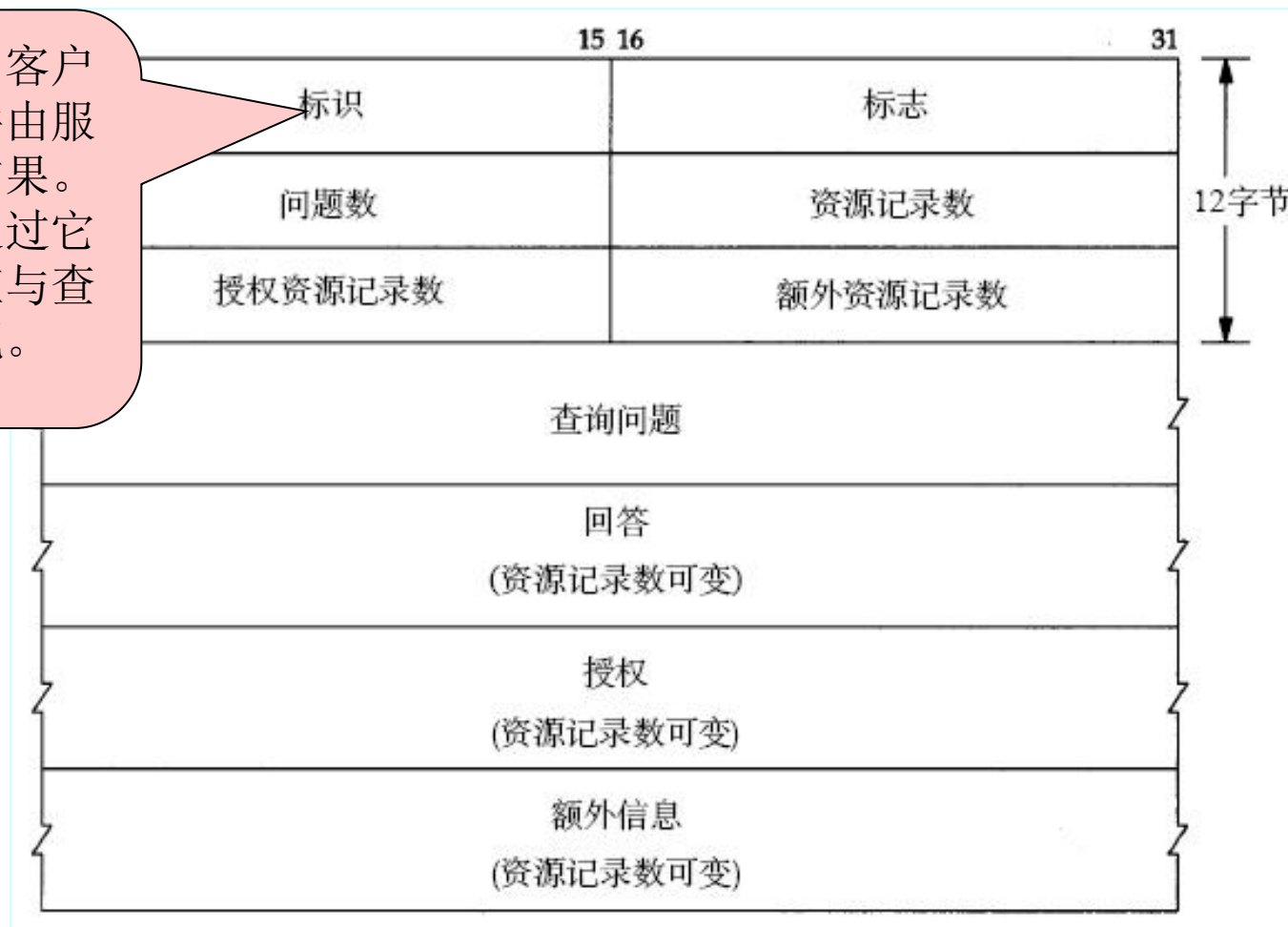


DNS面临的安全问题-事件ID

欺骗

- 通过伪装DNS服务器提前向客户发送与查询数据报ID相同的响应数据报，只要该伪造报文在真正响应

标识字段由客户程序设置并由服务器返回结果。客户程序通过它来确定响应与查询是否匹配。



冲



缓存中毒举例



- **A**向解析器**R**请求查询**XXX.baidu.com**的**IP**地址（假设合法地址为**IP1**）
- **R**会将查询请求转向**baidu.com**的授权域名服务器
- 攻击者会在响应到达**R**前伪造大量解析地址为**IP2**的响应包给**R**（需要成功猜测事务**ID**）
- **R**很难解除来源响应的真实性，且**DNS**会丢且第一个响应包后面的其他响应



IPsec的组成-协议组成



- IPsec把IP通信过程分为协商和数据交互两阶段
 - 协商阶段，通信双方互相认证对方身份，并根据安全策略使用的加密、认证算法，生成会话密钥
 - 数据交互阶段，双方利用协商好的算法和密钥对数据进行安全处理

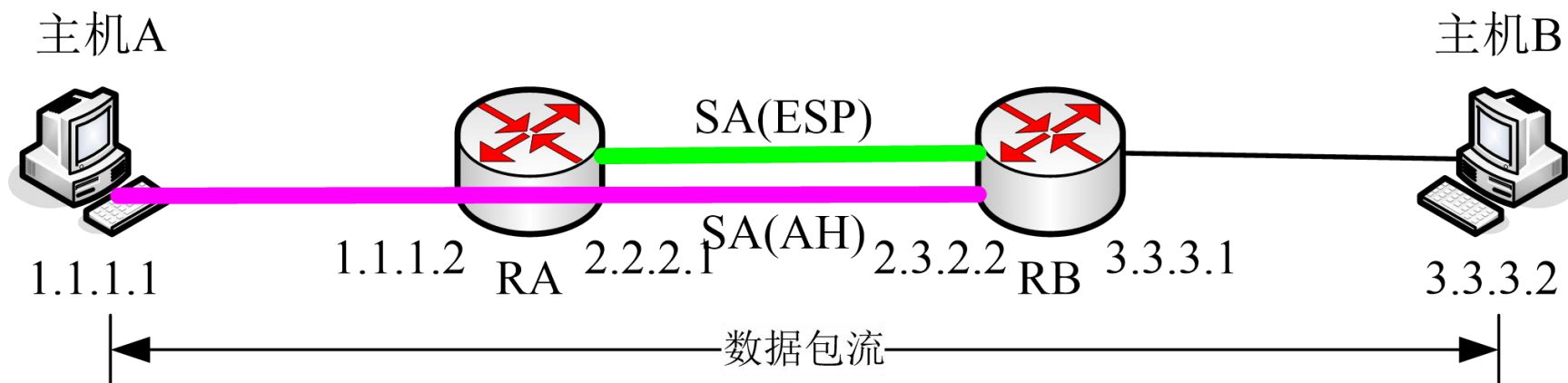


IPSec处理(1)—外出处理

- 外出处理过程中，传输层的数据包流入**IP**层。**IP**层检索**SPD**数据库，判断应为此包提供哪些服务。可能有以下几种情况：
 - 丢弃：简单丢掉；
 - 绕过安全服务：为载荷增添**IP**头，然后分发**IP**包；
 - 应用安全服务：假设已建立**SA**，则返回指向该**SA**的指针；如果未建立**SA**，则调用**IKE**建立**SA**。如果策略规定强行将**IPSec**应用于数据包，则在**SA**正式建立起来之前，包是不会被传送出去的。**SA**建好之后，会按正确顺序增添适当的**AH**和**ESP**头。

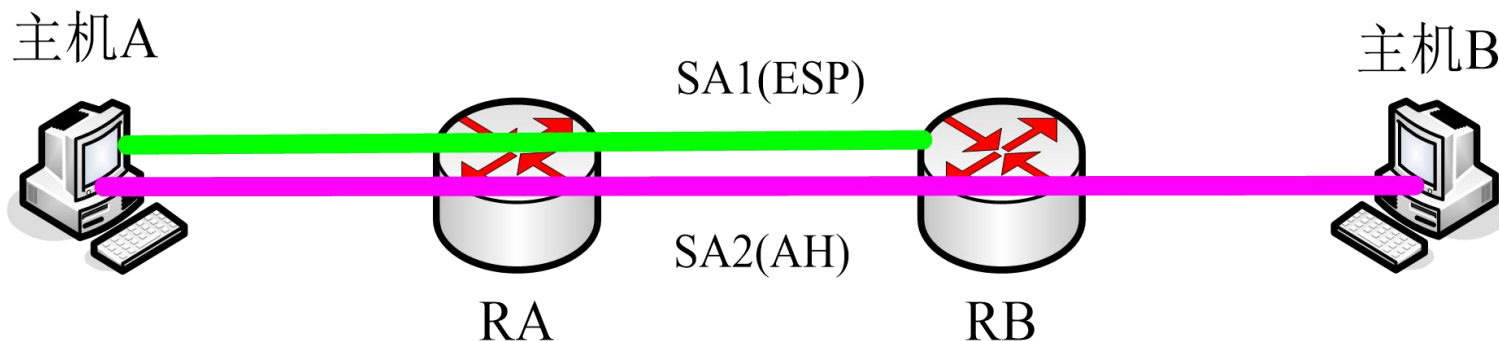


嵌套隧道



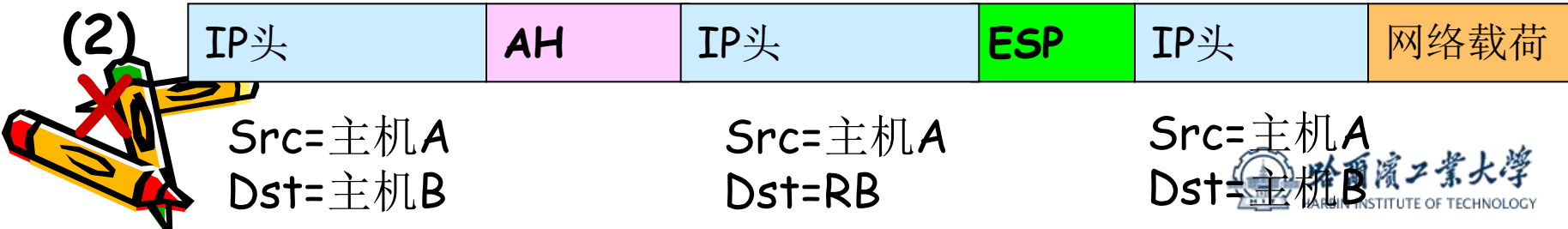
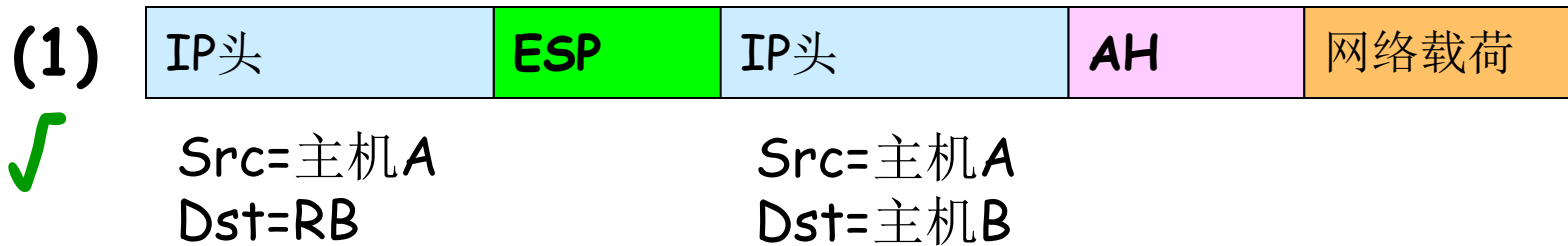
IP头	ESP	IP头	AH	IP头	数据
Src=2.2.2.1 Dst=2.3.2.2		Src=1.1.1.1 Dst=2.3.2.2		Src=1.1.1.1 Dst=3.3.3.2	





要求：主机A用ESP隧道将包传给RB，但要主机B验证自己。
处理：

- IKE建立4个SA：两个用于接收（暂不考虑），两个用于接收（SA1和SA2）；
- SA1：主机A和RB之间；SA2：主机A和主机B之间



IPSec处理(2)一进入处理

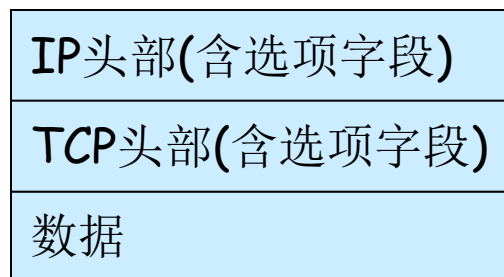


- 收到**IP**包后，如果包内没有**IPSec**头，则根据安全策略对包进行检查，决定如何处理：
 - 丢弃：直接丢弃；
 - 应用安全服务：**SA**没有建立，包同样会被丢弃；
 - 否则将包传给上层协议处理。
- 如果**IP**包中包含了**IPSec**包：
 - 从**IP**包中提取三元组(**SPI**，目标地址，协议)在**SAD**中检索。根据协议值交给**AH**层或**ESP**层处理。协议载荷处理完之后，要在**SPD**中查询策略，验证**SA**使用是否得当。

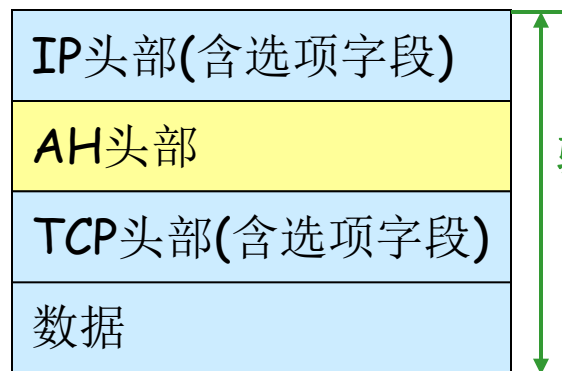


AH运行模式(1)—传输模式

- AH插入到IP头部（包括IP选项字段）之后，传输层协议（如TCP、UDP）或者其他IPSec协议之前。



图a 应用AH之前



验证区域
(可变字段除外)

图b 应用AH之后



AH与NAT冲突

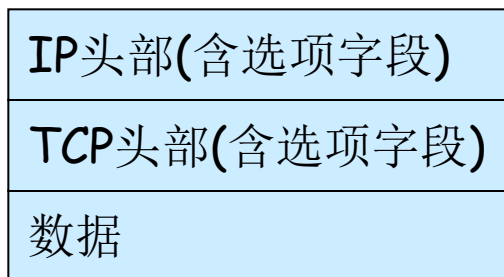


- 被AH验证的区域是整个IP包（可变字段除外），包括IP包头部，因此源IP地址、目的IP地址是不能修改的，否则会被检测出来。
- 然而，如果该包在传送的过程中经过NAT网关，其源/目的IP地址将被改变，将造成到达目的地址后的完整性验证失败。因此，AH在传输模式下和NAT是冲突的，不能同时使用，或者说AH不能穿越NAT。

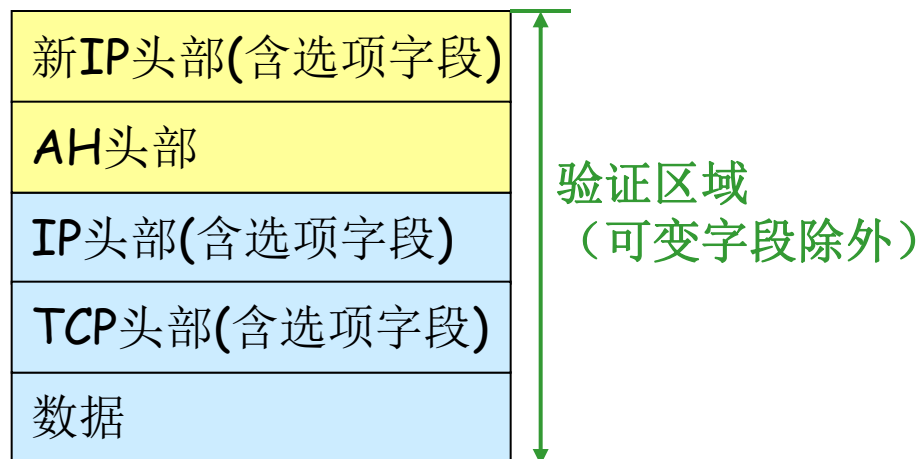


AH运行模式(2)——隧道模式

- 在隧道模式中，**AH**插入到原始**IP**头部之前，然后在**AH**之前再增加一个新的**IP**头部。



图a 应用AH之前



图b 应用AH之后

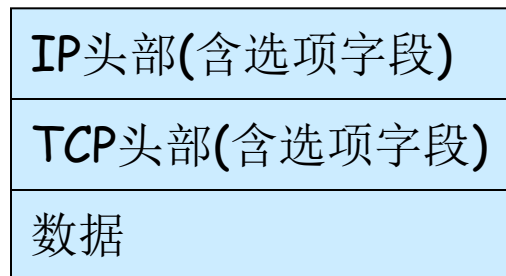




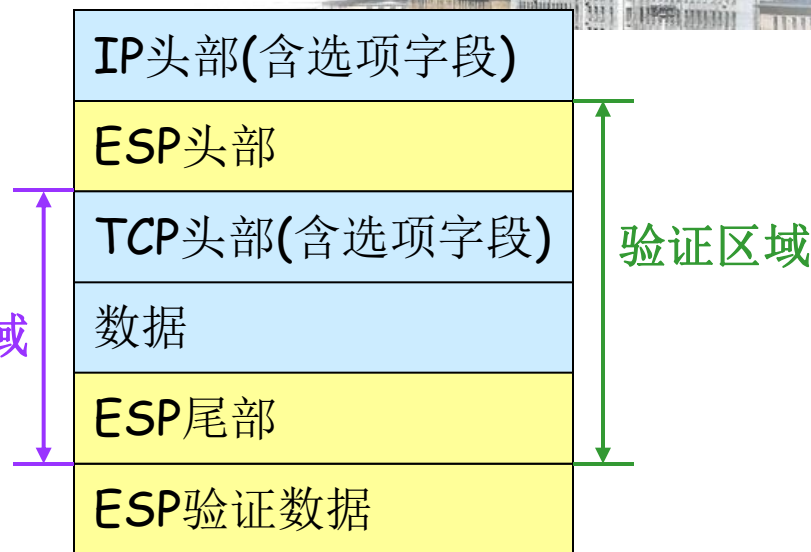
- 隧道模式下，**AH**验证的范围也是整个**IP**包，因此上面讨论的**AH**和**NAT**的冲突在隧道模式下也存在。
- 在隧道模式中，**AH**可以单独使用，也可以和**ESP**一起嵌套使用。



ESP传输模式示意图

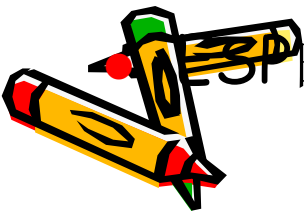


图a 应用ESP之前



图b 应用ESP之后

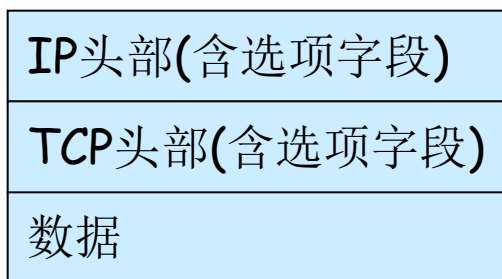
- 加密不包括SPI和序号字段;
- 加密不包括验证数据;



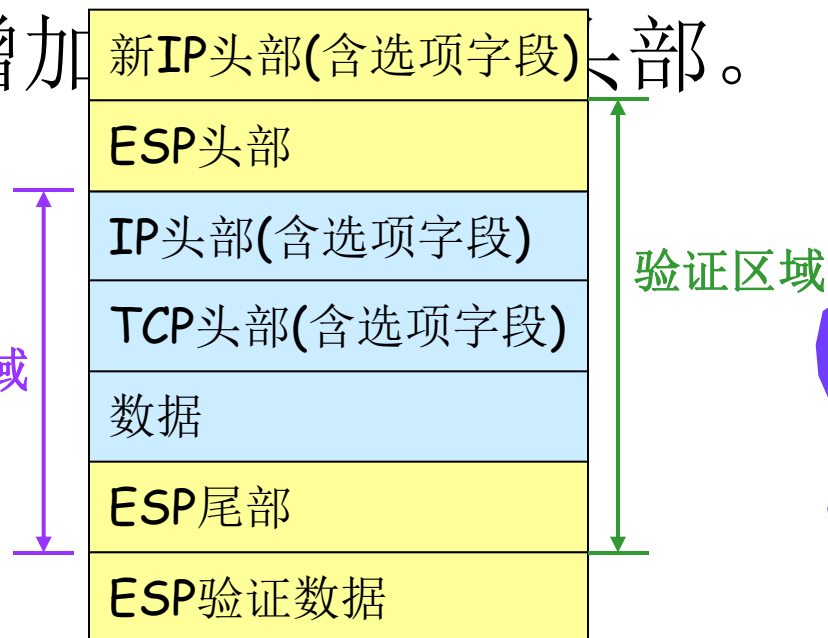
ESP的验证不包括IP头部, 所以不存在与NAT冲突的问题。

ESP运行模式(2)—隧道模式

- 保护的是整个IP包，对整个IP包进行加密；
- 在隧道模式中，ESP插入到原始IP头部之前，然后在ESP之前再增加新IP头部(含选项字段)头部。



加密区域



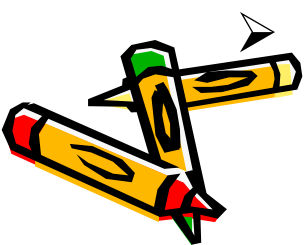
图a 应用ESP之前

图b 应用ESP之后

IKE交换模式



- 第一阶段
 - 主模式
 - 积极模式/野蛮模式
 - （建立**IKE SA**，建立验证过的密钥，是其他交换的前提条件）
- 第二阶段
 - 快速模式
 - （为**IPSec**协商安全服务）



• 第一阶段 准备工作

- 在前2条消息发送以前,发送者和接受者必须先计算出各自的**cookie**(可以防重放和**DOS**攻击),这些**cookie**用于标识每个单独的协商交换消息

-**cookie**---**RFC**建议将源目**IP**,源目端口,本地生成的随机数,日期和时间进行散列操作.**cookie**成为留在**IKE**协商中交换信息的唯一标识,实际上**cookie**是用来防止**DOS**攻击的,它把和其他设备建立**IPSEC**所需要的连接信息不是以缓存的形式保存在路由器里,而是把这些信息**HASH**成个**cookie**值





- 1&2消息

消息1---发送方向对等体发送一条包含一组或多组策略提议,在策略提议中包括5元组(加密算法,散列算法,DH,认证方法,IKE SA寿命)

消息2---接受方查看IKE策略消息,并尝试在本地寻找与之匹配的策略,找到后,则有一条消息去回应

注意!!!发起者会将它的所有策略发送给接受者,接受者则在自己的策略中寻找与之匹配的策略(对比顺序从优先级号小的到大的)(默认策略实际就是个模版没作用,如果认证只配置预共享的话,其他参数就会copy默认策略里的)





- 3&4消息

这2条消息,用于交换DH的公开信息和随机数

两个对等体根据DH的公开信息都算出了双方相等的密植后,两个nonce连同预共享密钥生成第一个skeyID

随后便根据SKEY__ID来推算出其他几个skeyID

skeyID_d---用来协商出后续IPSEC SA加密使用的密钥的

skeyID_a---为后续的IKE消息协商以及IPSEC SA协商进行完整性检查(HMAC中的密钥)

skeyID_e---为后续的IKE消息协商以及IPSEC SA协商进行加密





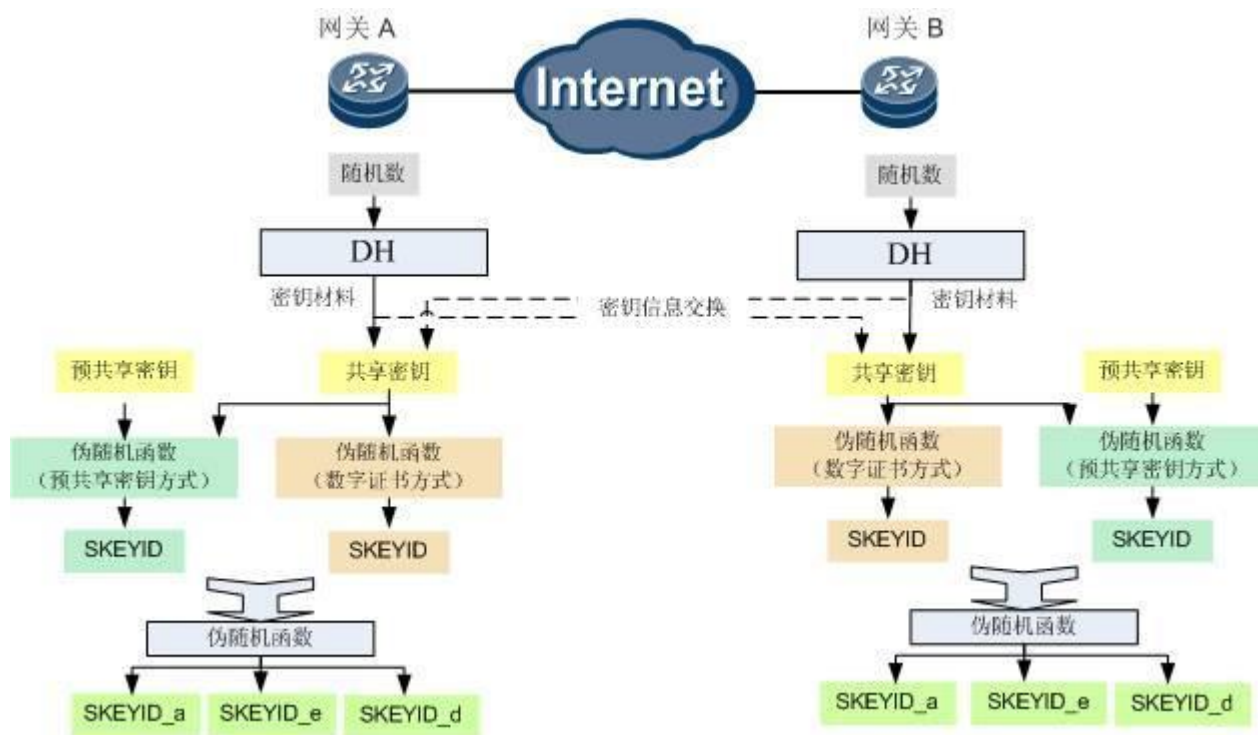
- 5&6消息

这2条消息用于双方彼此验证,这个过程是受skeyID_e加密保护的

为了正确生成密钥,每一个对等体必须找到与对方相对应的预共享密钥,当有许多对等体连接时,每一对对等体两端都需要配置预共享密钥,每一对等体都必须使用ISAKMP分组的源IP来查找与其对等体对应的预共享密钥(此时由于ID还没到,彼此先用HASH来彼此验证对方)

HASH认证成分——SKEYID_a,cookieA,cookieB, preshare_key,SA payload,转换集, 策略



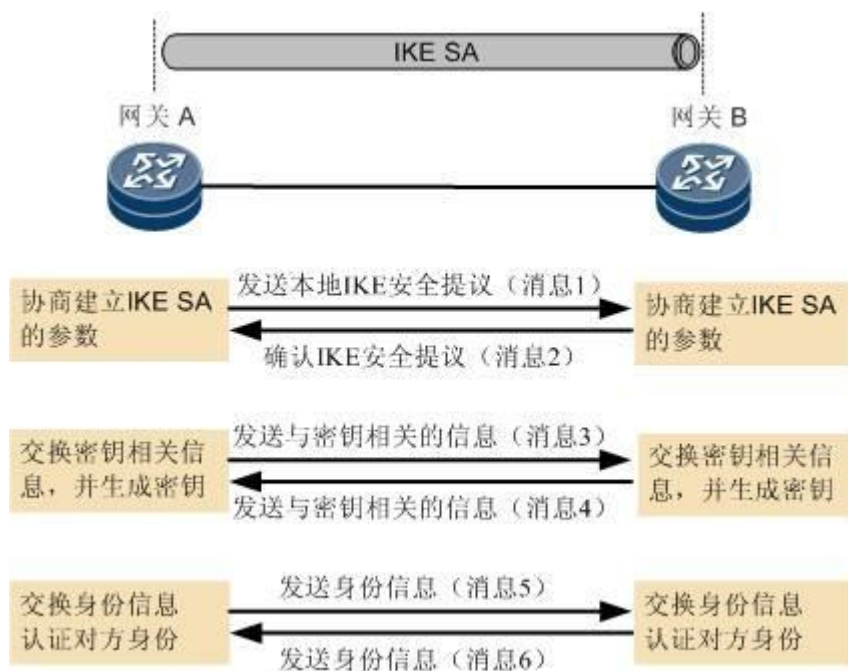


使用DH算法交换与密钥相关的信息，并生成密钥。

- 两个对等体通过两条ISAKMP消息（3、4）交换与密钥相关的信息。
- 由获得的密钥信息推导出4个密钥。其中SKEYID为基础密钥，通过它可以推导出SKEYID_a，为ISAKMP消息完整性验证密钥；可以推导出SKEYID_e，为ISAKMP消息加密密钥；可以推导出SKEYID_d，用于衍生出IPSec报文加密、验证密钥。

预共享密钥方式和数字证书方式下SKEYID的计算公式不同。





对等体之间验证彼此身份。

- 两个对等体通过两条**ISAKMP**消息（5、6）交换身份信息（预共享密钥方式下为**IP**地址或名称，数字证书方式下还需要传输证书的内容），身份信息通过**SKEYID_e**加密，故可以保证身份信息的安全性。
- 两个对等体使用**IKE**安全提议中定义的加密算法、验证算法、身份验证方法和**SKEYID_a**、**SKEYID_e**对**IKE**消息进行加解密和验证。

IKEv1支持以下几种身份验证方法：

- **预共享密钥**

这种方法要求对等体双方必须要有相同的预共享密钥（该密钥直接参与**SKEYID**的生成计算）。对于设备数量较少的**VPN**网络来说易于配置，在大型**VPN**网络中，不建议采用预共享密钥来做身份验证。

- **RSA**签名（通常称为数字证书）

数字证书需要由**CA**服务器来颁发。这种方法适用于大型动态的**VPN**网络。证书验证和预共享密钥验证的主要区别在于**SKEYID**的计算和交换的身份信息，其他的交换和计算过程和预共享密钥验证方式相同。

- **RSA**加密认证

此方法要求发起方已经知道响应方的公钥。发起方使用对方的公钥加密**nonce**和双方的身份**ID**来验证对方身份。此认证方法只能在**IKEv1**的主模式协商过程中使用，不能在**IKEv1**野蛮模式协商过程中使用。

- **数字信封认证**

设备支持**RSA**数字信封认证和**SM2**数字信封认证两种。

在数字信封认证中，发起方采用对称密钥加密信息内容，并通过非对称密钥的公钥加密对称密钥，从而保证只有特定的对端才能阅读通信的内容，从而确定对端的身份。



• 第二阶段(3条)

phase 2的目标是协商**IPSEC SA**,而且只有一种模式,快速模式,快速模式的协商是受**IKE SA**保护的

消息1---发送方发送一条报文,其中包含**HASH**,**IPSEC**策略提议,**NONCE**和可选的**DH**,身份**ID**

HASH:是用于给接受方作完整性检查的,用于再次认证对等体(必须)**HASH**的成分和5—6阶段一样

IPSEC策略提议:其中包括了安全协议,**SPI**,散列算法,隧道模式,**IPSEC SA**生命周期(必须)

NONCE:用于防重放攻击,还被用作密码生成的材料,仅当启用**PFS**时用到

ID:描述**IPSEC SA**是为哪些地址,协议和端口建立的

PFS(利用**DH**交换,可选):用了**PFS**后就会在第二阶段重新**DH**出个数据加密**KEY**,这个**KEY**和以前**IKE**协商出来的**KEY**没有任何关系,然后由这个新**KEY**来加密数据,只有到这个**IPSEC SA**的生命周期后,会再次**DH**出新的**KEY**,这样,安全性就提高了(普通等**ipsec SA**过期或密钥超时时,重新生成的数据加密密钥还是根据以阶段**DH**出来的**skeyID_d**衍生出来的)(**PFS**启用后,数据加密部分使用的密钥就没有了衍生的过程)

DH:重新协商**IPSEC SA**实使用的密钥(正常情况下**IPSEC**阶段使用的密钥都是由**skeyID_d**衍生而来,密钥之间都有一定的关系,就算**IPSEC SA**超时,新的**KEY**还是和**skeyID_d**有一定的关系)



• IPSec流程实例

- 为简单起见，我们假设这是一个**Intranet**例子，每台主机都有处于激活状态的**IPSec**策略：
- 1. 用户甲（在主机**A**上）向用户乙（在主机**B**上）发送一消息
- 2. 主机**A**上的**IPSec**驱动程序检查**IP**筛选器，查看数据包是否需要受保护以及需要受到何种保护
- 3. 驱动程序通知**IKE**开始安全协商
- 4. 主机**B**上的**IKE**收到请求安全协商通知
- 5. 两台主机建立第一阶段**SA**，各自生成共享"主密钥" 注：若两机在此前通信中已经建立起第一阶段**SA**，则可直接进行第二阶段**SA**协商



- 6. 协商建立第二阶段**SA**对：入站**SA**和出站**SA**。**SA**包括密钥和**SPI**
- 7. 主机**A**上**IPSec**驱动程序使用出站**SA**对数据包进行签名（完整性检查）与/或加密。
- 8. 驱动程序将数据包递交**IP**层，再由**IP**层将数据包转发至主机**B**
- 9. 主机**B**网络适配器驱动程序收到数据包并提交给**IPSec**驱动程序。
- 10. 主机**B**上的**IPSec**驱动程序使用入站**SA**检查完整性签名与/或对数据包进行解密。
- 11. 驱动程序将解密后的数据包提交上层**TCP/IP**驱动程序，再由**TCP/IP**驱动程序将数据包提交主机**B**的接收应用程序。
- 以上是**IPSec**的一个完整工作流程，虽然看起来很复杂，但所有操作对用户是完全透明的。中介路由器或转发器仅负责数据包的转发，如果中途遇到防火墙、安全路由器或代理服务器，则要求它们具有**IP**转发功能，以确保**IPSec**和**IKE**数据流不会遭拒绝。

• 这里需要指出的一点是，使用**IPSec**保护的数据包不能通过网络地址译码**NAT**。因为**IKE**协商中所携带的**IP**地址是不能被**NAT**改变的，对地址的任何修改都会导致完整性检查失效。



PKI系统主要组成

- 认证机构CA (Certificate Authority)
 - 证书的签发机构
- 证书库
 - 证书的集中存放地，提供公众查询
- 密钥备份和恢复系统
 - 备份用户的解密密钥，以便在丢失后恢复
- 证书撤销处理系统
 - 根据需要作废证书
- PKI应用程序接口
 - 为各种应用提供访问PKI的方式



认证机构CA



- 权威的、可信任的、公正的第三方机构
- 认证中心
- **PKI**的核心
- 数字证书的签发机构
- 证书的产生、管理、存档、发放、以及作废管理



CA的任务



- 验证并标识证书申请者的身份
- 审批、发放、更新、验证证书
- 证书资料信息的管理（公钥证书序列号、**CA**标识）
- 产生和发布撤销证书列表
- 证书的归档
- 密钥归档
- 历史数据归档



CA的职责



- 确保**CA**用于签名证书的非对称密钥的质量
- 确保整个签证过程的安全性，确保签名私钥的安全性
- 确保证书主体标识的唯一性，防止重名



证书



- 公钥体制的一种密钥管理媒介
- 权威性的电子文档，网络身份证
- 证明主体身份及其公钥的合法性
- 含有主体的身份和公钥
- 用**CA**的私钥签名



证书主体公钥的产生方式

- 主体用户自己生成密钥对
 - 将公钥传送给**CA**
 - 要保证公钥的可验证性和完整性
- **CA**替用户生成密钥对
 - 将私钥传递给用户
 - 要保证私钥的机密性、可验证性和完整性



证书的撤销



- 证书废止原因
 - 私钥泄密
 - 从属变更
 - 终止使用
 - **CA**出现问题
- 撤销证书原因
 - **CA**知道证书细节不真实
 - 证书持有者没有履行职责和登记人协议
 - 证书持有者死亡、违反电子交易规则或者犯罪



撤销机制



- **CA**公布撤销证书列表**CRL**
 - 被撤销证书的序列号
 - 撤销日期
 - 撤销原因
- 撤销证书列表**CRL**由**CA**私钥签名
 - 保证完整性
 - 保证权威性
- **CRL**公布在目录中
- **CRL**公布周期由**CA**决定



CA的组件构成



- 认证操作管理规范CPS
- 注册机构RA
- 证书服务器
- 证书库
- 密钥恢复服务
- 时间服务器
- 签名服务器



证书验证



- 护照的验证

- 查对照片、签名
- 检查全息防伪图片，确认护照真假
- 检查护照的有效期
- 核实条形码中的信息和护照的物理信息是否一致
- 在计算机上调出记录检查护照是否作废或被撤销



数字证书验证



- 验证证书签名者的签名
- 检查证书的有效期
- 检查证书的预期用途是否符合**CA**在该证书中指定的策略限制
- 确认证书没有被**CA**撤销



使用证书的应用程序

- 安全套接字层SSL
 - <https://www.abcd.com>
- 安全的电子邮件
- 虚拟专用网



安全套接字层SSL



- <https://www.abcd.com>
- 将**Web**浏览器定向到**Web**服务器的一个另外不同的端口上
- 会话中所有信息都被加密
- 用证书确定身份
 - 服务器端认证: **Web**服务器发送一个证书给**Web**浏览器供其认证
 - 客户端认证: **Web**服务器请求用户的证书进行认证



安全的电子邮件



- 数字签名
 - 发送者用自己的私钥创建签名，将相应的证书同邮件一起发送
- 保密
 - 发送者先获得接收者的证书，检验合格后用其中的公钥加密邮件发送



虚拟专用网



- 像使用私有网络一样使用公共网络
- 信息必须加密
- 必须确定与之通信的机器的身份
- 每个终端实体向对方提交自己的证书并证明自己确实拥有相应的私钥
 - 证书中含有作为主体标识的网络地址



证书分发

- 在协议内部分发证书
- 使用库分发证书



在协议内部分发证书

- 电子邮件签名
 - 与签名者私钥相应的证书、公钥和电子邮件消息一起发送
 - 在认证路径中证书的其余部分也被发送
- 客户端**SSL (secure socket layer)**认证
 - 浏览器或者其他客户应用在服务器要求下通过协议发送客户证书



使用库分发证书



- 电子邮件加密
 - 访问**LDAP**目录
 - 地址簿中的每个用户条目连接与用户对应的**LDAP**目录项
- 服务器端**SSL**认证
 - 在本地证书库中找到**CA**证书
 - 使用目录服务器查询**CA**证书



攻击技术分析-拒绝服务攻击 (DoS)



- 拒绝服务攻击分为两类
 - 由于错误的配置或者软件的弱点导致的，这类攻击可以通过开发商发布的补丁来解决
 - 利用合理的服务请求来占用过多的服务资源，致使服务超载，无法响应其他请求。



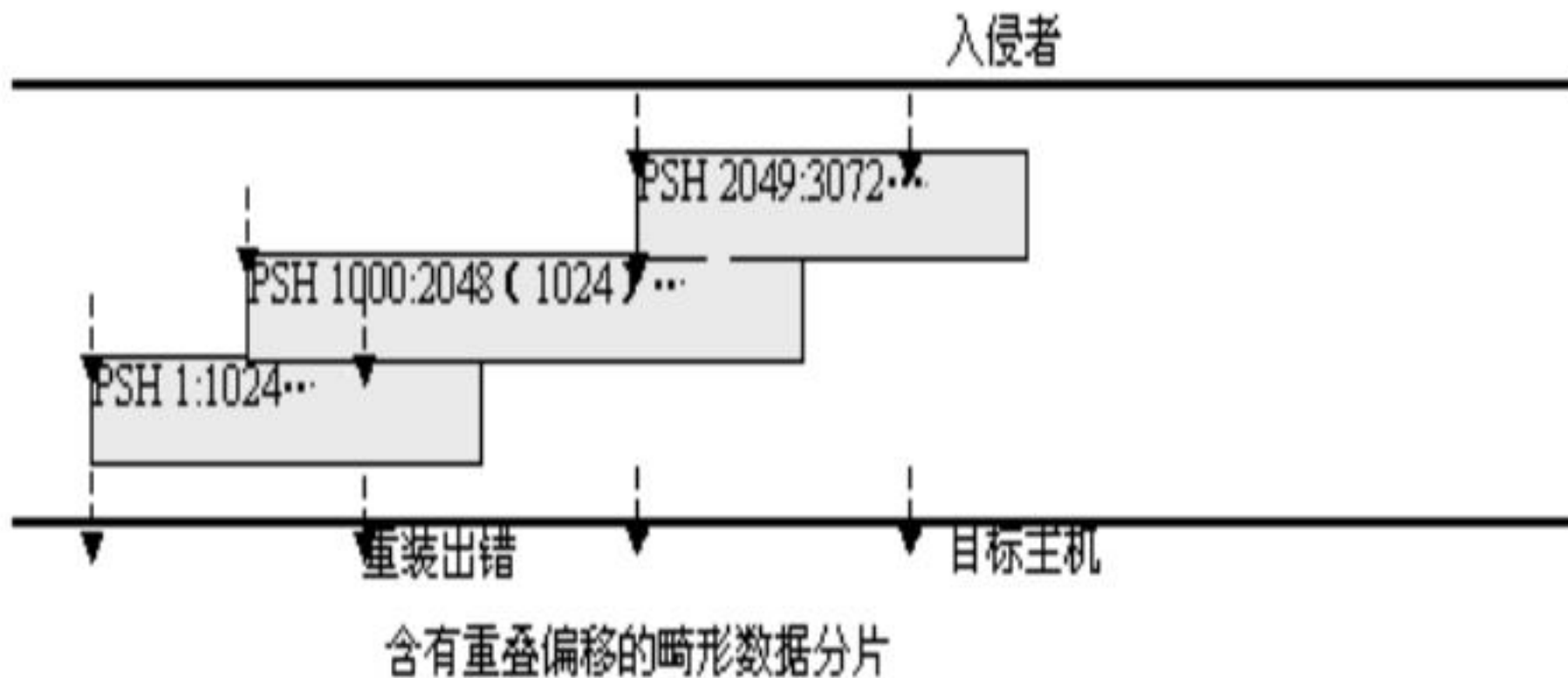
2、泪滴(Teardrop) 攻击(1/7)

- 原理：利用异常的数据分片导致接收方在处理分片数据时崩溃，也称为“**碎片攻击**”。
 - Teardrop本是一段用于DoS攻击的程序名，该程序利用Windows95/NT/3.1和低版本的Linux中处理IP分片的漏洞，向受害者发送偏移地址重叠的UDP数据包分片，使得目标机器在将分片重组时出现异常错误，导致目标系统崩溃或重启



2、泪滴(Teardrop) 攻击(4/7)

攻击者发送畸形数据片段



2、泪滴(Teardrop) 攻击(6/7)

- 利用数据报分片的其他攻击（变种二）：重叠分片攻击，目的不是DoS，而是穿透防火墙。

➢ 原理：防火墙在处理重叠分片时与终端系统（这里指受害者最终目标主机系统）之间可能存在差异性：当重叠时，有的系统是以先到的数据为主，有的系统则是用后到的数据覆盖先前的数据。

攻击数据穿过防火墙的前提条件：防火墙重组数据时与受害者主机不同

➢ 例子：假设受害者主机以后到的数据优先原则处理TCP协议，第一个分片中的TCP服务类型设为端口80(HTTP)，这是大多数防火墙允许的，而在第二个分片中以端口23(Telnet)改写第一个分片中的80端口，从而穿过了防火墙！



2、泪滴(Teardrop) 攻击(7/7)

- 利用数据报分片的其他攻击（变种三）：通过分片导致防火墙或IDS的拒绝服务。

➤ 原理：有些防火墙或IDS为了检测碎片攻击或其他类型的利用分片的攻击而设置了碎片重组：当收到分片数据包时并不单独检测，而是等待所有的分片都到达（必须缓存已到的分片），重组完成后再检测。攻击者如何利用这一点？

➤ 攻击：攻击者伪造并发送大量的分片，但却不让这些分片构成完整的数据报以此占用防火墙或IDS的CPU和存储单元，构成DoS攻击。



3、Land 攻击



- Land攻击
- 原是一段C程序，其功能是向受害者发送TCP SYN包，而这些包的源IP地址和目的IP地址被伪造成受害者的IP地址，源端口和目的端口也是相同的（端口必须是激活的？），目标系统在收到这样的包以后可能会挂起、崩溃或重启。Why? Why? Why?
- TCP SYN包：连接请求
- 导致接收服务器向它自己的地址发送SYN-ACK消息，结果这个地址又发回ACK消息并创建一个空连接，每一个这样的连接都将保留直到超时掉，对Land攻击反应不同，许多UNIX实现将崩溃，NT变的极其缓慢（大约持续五分钟）
- 前提条件：对应的端口所提供的服务器必须是激活的！



5、循环攻击

- 也称：**振荡攻击**(Oscillate Attack)或乒乓攻击，其原理

:

- 当两个都会产生输出的端口(可以是一个系统 / 一台机器的**两个端口**，也可以是不同系统 / 机器的两个端口)之间建立连接以后，第一个端口的输出成为第二个端口的输入，导致第二个端口产生输出，同时第二个端口的输出又成为第一个端口的输入，如此，一两个端口间将会有大量的数据包产生，导致拒绝服务

- 典型攻击：

- **Echo Chargen**攻击：当运行着**Chargen**服务的UDP端口(19)收到一个数据包后，会产生一个字符串作为回应。**Echo**服务的UDP端口(7)收到一个数据包原样返回，这两种服务可被攻击者用来进行循环攻击。**How?**





- 攻击者只需向**A**系统的**UDP Echo**端口发送一个好像来自于**B**系统的**UDP Chargen**端口的**UDP**包（即假冒**B**系统的**IP**地址和**Chargen**端口），或向**B**系统的**UDP Chargen**端口发送一个好像来自**A**系统的**UDP Echo**端口的数据包，则在**AB**系统的这两个端口间将来回不停地产生**UDP**包



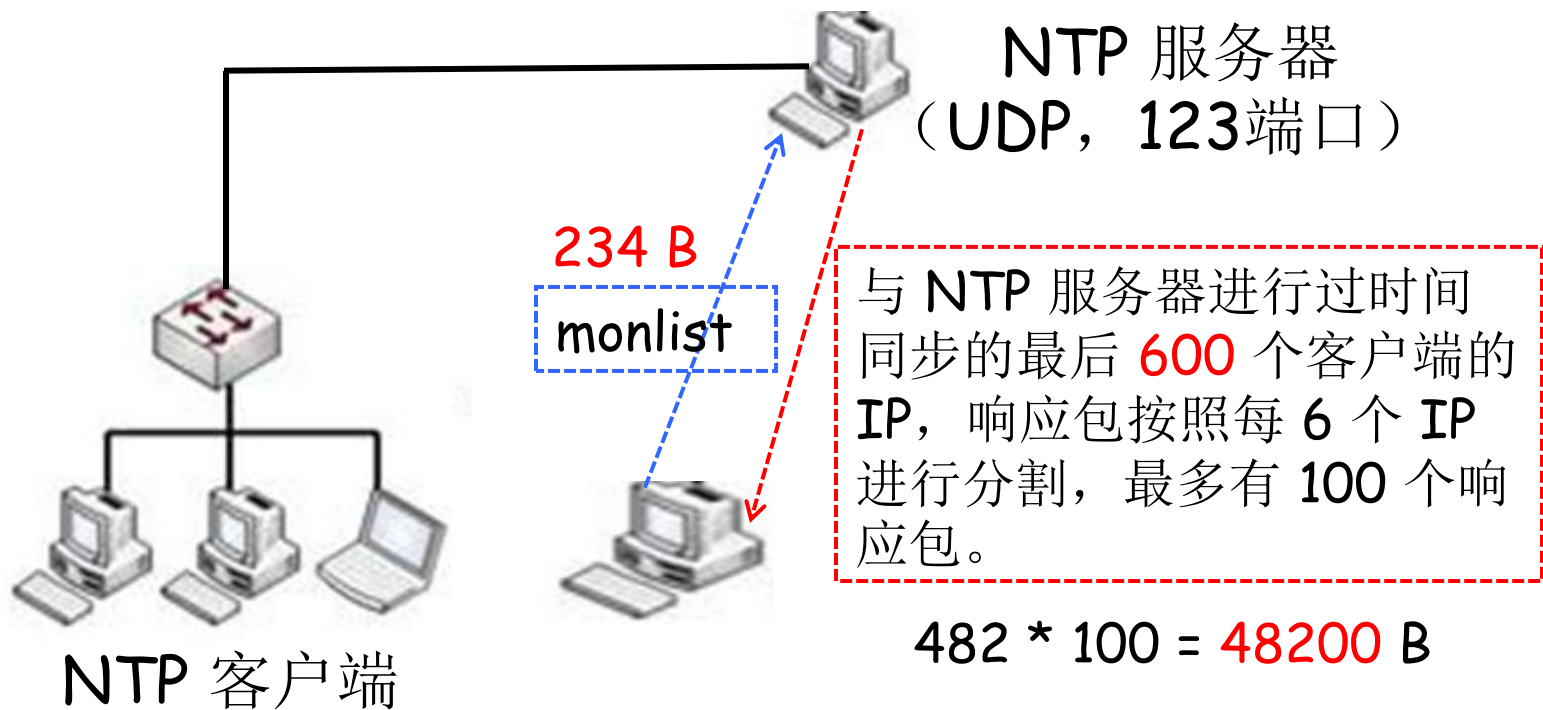
2、SYN风暴攻击(直接型)

- 原理：发送大量**SYN**报文，但对服务器的**SYN+ACK**应答报文不作应答，即三次握手的第三次握手无法完成，造成服务器维护大量的**半连接列表**，消耗服务器半连接资源（**一般系统的上限为1024，超过此数则不接受新的连接请求**）的攻击方式。
 - 需伪造地址，一方面逃避追踪，另一方面为了攻击能成功。
Why? Why?
 - 如果不伪造地址，在未修改攻击主机**TCP**协议栈的情况下，其系统会自动对**SYN-ACK**做出响应，无论是其以**ACK**回应建立连接还是以**RST**回应取消连接都会在服务器上释放对应的半连接（一般会回应**RST**）从而影响攻击主机的性能，又由于攻击主机发出的响应使得受害者较早地释放对应的半开连接进而减少占用半开连接的时间，降低攻击效果。如果用的是假地址（根本不存在或没开机），则受害者必须等待超时才能释放相应的半开连接。



NTP DDoS (DRDoS)

- NTP: 利用monlist进行攻击

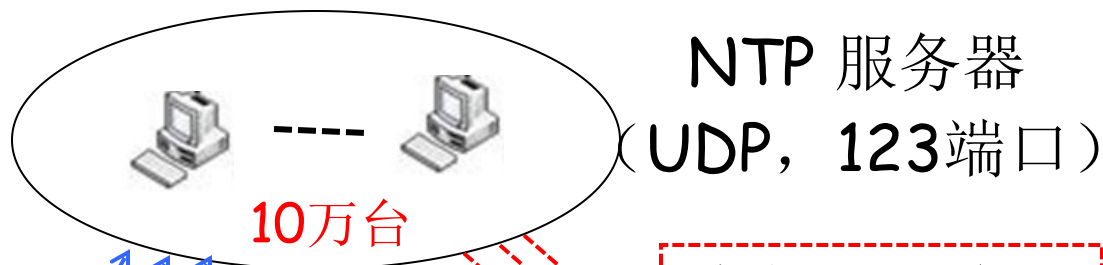


- 1、回复报文长度是申请报文长度的: $482 * 100 / 234 = 206$ 倍
- 2、使用UDP协议, 很容易伪造源地址给NTP服务器发请求。

NTP DDoS (DRDoS)

- NTP: 利用monlist功能进行DDoS攻击

CNVD-2014-00082



来自“受害者”
的Monlist请求

来自NTP服务器的
Monlist回复

受害者

攻击者

$$\begin{aligned} & 10 \text{ (请求 / 秒)} * 48200 \text{ B / 回复} * \\ & 100000 \text{ 台} * 8 \text{ bit} = 3856 \times 10^8 \text{ bps} \\ & = 385.6 \text{ Gbps} \end{aligned}$$

SSDP DDoS攻击的 6 个步骤

- 首先，攻击者进行扫描，寻找可以用作放大因子的即插即用设备。
- 攻击者发现联网设备后，创建所有发出响应的设备的列表。
- 攻击者使用目标受害者的欺骗性 IP 地址创建 UDP 数据包。
- 然后，攻击者使用僵尸网络通过设置某些标志（比如 **ssdp:rootdevice** 或 **ssdp:all**），向每个即插即用设备发送一个欺骗性发现数据包，并请求尽可能多的数据。
- 结果，每个设备都会向目标受害者发送回复，其数据量最多达到攻击者请求的 30 倍。
- 然后，目标从所有设备接收大量流量，因此不堪重负，可能导致对正常流量拒绝服务。



僵尸网络

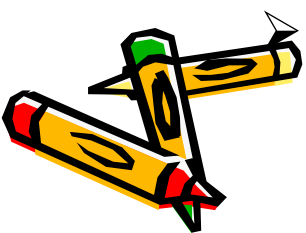


- 两种典型僵尸网络架构
 - 基于**IRC**的僵尸网络
 - 基于**P2P**的僵尸网络



拒绝服务攻击的作用

- 首要作用：瘫痪目标
- 其它作用呢？
 - SYN Flood攻击可以用于IP劫持、IP欺骗等
 - 使某些安全设备（如防火墙）失效
 - 重启系统，使漏洞利用、更改的配置生效、提升权限
 - 瘫痪目标后，假冒目标（如DNS）



（一）检测（3/4）

• 根据异常流量来检测（Cont）

- DoS工具产生的网络通信信息有两种：**控制信息**（在DoS管理者与攻击代理之间）和**攻击时的网络通信**（在DoS攻击代理与目标主机之间）。根据以下异常现象在入侵检测系统中建立相应规则，能够较准确地监测出DoS攻击
- 1. 大量的目标主机域名解析
 - 根据分析，攻击者在进行DDoS攻击前总要解析目标的主机名。**BIND**域名服务器能够记录这些请求。由于每台攻击服务器在进行一个攻击前会发出**PTR**反向查询请求，也就是说在DDoS攻击前域名服务器会接收到大量的反向解析目标**IP**主机名的**PTR**查询请求。



(一) 检测 (4/4)

- 根据异常流量来检测:

➤ 2. 极限通讯流量

- 当DDoS攻击一个站点时，会出现明显超出该网络正常工作时的极限通信流量的现象。现在的技术能够分别对不同的源地址计算出对应的极限值。当明显超出此极限值时就表明存在DDoS攻击的通信。因此可以在主干路由器端建立ACL访问控制规则以监测和过滤这些通信。

➤ 3. 特大型的ICMP和UDP数据包

- 正常的UDP会话一般都使用小的UDP包，通常有效数据内容不超过10字节。正常的ICMP消息长度在64到128字节之间。那些长度明显大得多的数据包很有可能就是DDoS攻击控制信息，主要含有加密后的目标地址和一些命令选项。一旦捕获到（没有经过伪造的）控制信息，DDoS服务器的位置就暴露出来了，因为控制信息数据包的目标地址是没有伪造的。

• 。



一) 检测 (4/4)



- 4. 不属于正常连接通信的**TCP和UDP数据包**。隐蔽的**DDoS**工具随机使用多种通信协议（包括基于连接的和无连接协议）发送数据。优秀的防火墙和路由规则能够发现这些数据包。
- 5. 数据段内容只包含文字和数字字符（例如，没有空格、标点和控制字符）的数据包。这往往是数据经过**BASE64**编码后而只会含有**BASE64**字符集字符的特征。**TFN2K**发送的控制信息数据包就是这种类型的数据包。**TFN2K**（及其变种）的特征模式是在数据段中有一串**A**字符（**AAA.....**），这是经过调整数据段大小和加密算法后的结果。如果没有使用**BASE64**编码，对于使用了加密算法数据包，这个连续的字符就是空格。



（二）响应

- 到目前为止，对付风暴型DDoS攻击的方案主要有四种：
 - 通过丢弃恶意分组的方法保护网络；
 - 在源端控制DDoS攻击；
 - 追溯 (Traceback) 攻击的源端, 然后阻止它发起新的攻击；
 - 路由器动态检测流量并进行控制。
- 最有效的对抗风暴型DDoS的方法是：**流量清洗**。



流量清洗

- DDoS攻击防御就是对DDoS攻击与正常业务数据混合在一起的流量进行净化，净化掉DDoS攻击流量，保留正常业务流量，保证客户业务7×24小时的不间断提供。
- DDoS攻击阻断过程一般包括攻击监测和判断、流量牵引、清洗过滤、流量回送四个关键环节。



（三）防范

- 限制带宽
 - 限制特定协议占用的带宽，但并不是完善的方法
- 终端防御：
 - 及时安装厂商补丁，减少被攻击的机会
 - 运行尽可能少的服务
 - 增强容忍性
- 入口过滤：只允许必要的通信
 - 设置严格的防火墙策略
 - 封锁所有无用的数据
- 完全阻止是不可能的，防范可减少被攻击的机会



多用户权限



- **Linux**是一个多用户系统，**Linux**将全部的特权授予了一个称为**Root**的管理人员用户。
- 其它用户可以属于不同的组
- 文件的权限既是属于用户的也是属于组的，所以除非其他用户被明确的授权否则他们不能读你的文件
 - **-Rw-rx-wx**
- 某些情况下需要允许某个非特权用户完成某一系统功能，而这一功能需要**root**特权



进程的识别号(ID)

- 真实用户识别号 (real user ID)
- 真实用户组识别号 (real group ID)
- 有效用户识别号 (effective user ID)
- 有效用户组识别号 (effective group ID)
- 一般情况真实用户ID和有效用户ID相同，都是运行该进程的用户ID
- 当设置了set-user-ID标志位时，真实用户ID仍是运行进程用户的ID,而有效用户ID变成了当前运行文件的所有者的ID
- 组识别号也是这样，区别在于设置了set-group-ID标志位



一个例子

- **Tip**是一个通过调制解调器与远程系统连接的程序，当**tip**使用一个调制解调器时，它必须通过使用锁文件来独占使用它。此锁文件是与**UUCP**程序共享的，因此这两个程序可能在同时要使用同一调制解调器



工作步骤

- **tip**程序文件是由用户uucp拥有的，并且其设置-用户-ID位已设置。
当**exec**此程序时，则关于用户ID得到下列结果：
实际用户ID=我们的用户ID
有效用户ID=UUCP
保存设置用户ID=UUCP
- **tip**存取所要求的锁文件。这些锁文件是由名为UUCP的用户所拥有的，因为有效 用户ID是UUCP，所以**tip**可以存取这些锁文件。
- **tip**执行**setuid(getuid())**。因为**tip**不是超级用户进程，所以这仅仅改变有效 用户ID。此时得到：
实际用户ID=我们的用户ID(未改变)
有效用户ID=我们的用户ID(改变)
保存设置用户ID=UUCP(未改变)
现在，**tip**进程是以我们的用户ID作为其有效用户ID而运行的。
这就意味着能存取的只是我 们通常可以存取的，没有附加的存取权。



工作步骤

- 当执行完我们所需的操作后，**tip**执行 **setuid(uucpuid)**，（其中**uucpuid**是用户**uucp**的**ID**号）。以为**uucpid**等于保存用户**ID**，符合规则，因此该调用正确执行现在得到：

实际用户**ID**=我们的用户**ID**(未改变)

有效用户**ID**=**uucp**

保存设置用户**ID**=**uucp**(未改变)

- 我们又取得了文件的访问权





- 授予用户**root**特权
- 赋予程序一种能力，使得程序象**root**用户的程序一样运行。正确的执行系统功能，而又不真正授予用户整个系统的控制权。
- 这种类型的权限称作**suid**。
- 任何用户在执行拥有**suid**权限的程序时，用户的**eu**id被改成程序所有者的**uid**，程序被执行。执行完成后，用户的**eu**id恢复为原来的值。





- 如果用户想修改自己的密码，他可以运行 `/usr/bin/passwd`，这是属于 `root` 的，且 `suid` 位打开。
- 将 `suid` 权限打开并且属于 `root` 用户的程序称为 `suid root` 程序
- 如果程序的流程可以改变执行一段注入的随意代码，则攻击者可以象 `root` 用户一样使程序作任何事



缓冲区溢出



- 缓冲区溢出攻击，是通过重写堆栈中储存的**EIP**地址的内容，以使程序跳转到指定**shellcode**处执行
- 主要是由一些不好的编程习惯，比如使用不安全的**strcpy**和**strcat**等



缓冲区溢出-堆栈的作用

- 堆栈在子程序调用的时候用来传递参数
- 堆栈用来保存子程序的返回地址
- 堆栈用来保存子程序中的变量





- **Nop**填充 降低计算返回地址的难度
- **Shellcode** 衍生出需要的**shell**
- 重复返回地址填充 增加覆盖原来返回地址的概率

Nop sled	Shellcode	重复返回地址
----------	-----------	--------



Printf工作原理

- 格式化函数每次遍历格式化串的一个字符。如果该字符不是格式化参数，则辅之输出该字符。如果遇到一个格式化参数，就采取相应的动作，并使用栈中与那个参数相对应的参量



格式化字符串的漏洞

- 格式化字符串漏洞同其他许多安全漏洞一样是由于程序员的懒惰造成的。程序员的任务是：打印输出一个字符串 `printf("%s", str);`
- 有时程序员为了节约时间和提高效率，并在源码中少输入6个字节，他会这样写：
`printf(str);`
- 错误的源码使得系统将用户的输入作为了具有控制功能的格式化串，从而可以通过构造输入来进行攻击



入侵检测简介

- 什么是入侵检测？
 - 入侵检测是用于检测任何损害或企图损害系统的保密性、完整性或可用性行为的一种网络安全技术。
- 手段
 - 通过监视受保护系统的状态和活动，采用误用检测或者异常检测的方式，发现非授权的或恶意的系统及网络行为。
- 前提
 - 非法行为和合法行为是可区分的
- 入侵监测需要解决的两个问题
 - 如何充分并可靠的提取描述行为特征的数据
 - 如何根据特征数据高效并准确地判定行为的性质



入侵检测系统术语



- 警报

- **IDS**向系统操作人员发出入侵正在发生或正在尝试进行的消息。

- 异常

- 用一段时间建立一个主机或者网络活动的轮廓。当一个用户行为或者网络行为与此轮廓距离超过某一个值的时候，需要发出警报，此行为称之为异常



入侵检测系统术语



- 网络入侵特征数据库
 - 将网络入侵行为抽象成特定的字符集和，通过与网络上或主机上的行为向匹配，发现可能的网络入侵。是基于误用检测系统的重要组成部分。
- 构造数据包
 - 不遵循通常的数据包结构，通过构造自己的数据包，进行数据包欺骗，或者是接受者无法处理这样的数据包



入侵检测系统术语



- 自动响应
 - 一些**IDS**能够对攻击做出防御性反应
 - 重新配置路由器或者防火墙，拒绝来自相同地址的流量
 - 发送**reset**包切断连接
 - 攻击者可以通过信任地址实施攻击，引起设备重新配置，达到拒绝服务攻击的目的
- 蜜罐（**Honeypot**）
 - 模拟存在漏洞的系统，为攻击者提供攻击目标。其在网络中没有任何用途，因此任何连接都是可能的攻击。
 - 诱惑攻击者在上面浪费时间，延缓对真正目标的攻击



入侵检测系统术语



- 安全策略
 - 事先定义的正式的文档声明，定义哪些服务可以通过被监控的网段，以支持组织的安全需求。它包括（但不仅限于）哪些主机不允许外部网络访问
- 感应器
 - 入侵检测构件，从数据源搜集数据。数据搜集的频率由具体提供的**IDS**决定



入侵检测的分类

- 从入侵检测的策略来看，入侵检测模型主要有两种：即滥用检测和异常检测。
- 滥用检测（**Misuse Detection**）是对利用已知的系统缺陷和已知的入侵方法进行入侵活动的检测。滥用检测的优点是可以有针对性地建立高效的入侵检测系统，其精确性较高，主要缺陷是不能检测未知的入侵，也不能检测已知入侵的变种，因此可能发生漏报。
- 异常入侵由用户的异常行为和对计算机资源的异常使用产生。异常检测（**Anomaly Detection**）需要建立目标系统及其用户的正常活动模型，然后基于这个模型对系统和用户的实际活动进行审计，以判定用户的行为是否对系统构成威胁。



基于主机的数据源



- 操作系统审计纪录
 - 由专门的操作系统机制产生的操作系统事件纪录
- 系统日志
 - 由系统程序产生的用于记录系统或应用程序事件的文件，通常以文本文件方式存放



基于主机的数据源-操作系统审计日志

- 由包含在操作系统软件内部的专门的审计子系统产生
- 用于反映系统活动的信息集合，按照时间组织成一个或者多个审计文件
- 每个审计文件由审计记录组成



审计日志存在的问题

- 系统开发商为了满足**C2**级，在各自的产品中集成了大量针对性很强的审计功能
- 审计记录的不兼容性
- 有些审计记录包含大量的无关信息
- 有些过于精简，同时缺乏审计子系统的说明文档



审计记录的特点



- 操作系统本身为审计子系统提供了实质性的保护，提高了信源的可信度
- 审计子系统从操作系统层次上获取信息，可以得到系统事件的细节，为入侵检测实现精确的模式匹配奠定基础
- 入侵者很难改变审计日志



系统日志



- 系统日志是反映各种系统事件和系统设置的文件
- 系统日志通常存储在系统未经保护的目录中，并采用文本方式存储
- 产生系统日志的软件通常作为应用程序运行，相对容易遭到篡改
- 对需要监视的主机的日志进行备份



系统日志



- 相对更加直观和人性化
- 在某些环境下，可能无法得到或者解释审计日志
- 使得入侵检测系统能过从不同的角度反映同一件事情



基于网络的数据源



- 通过网络监听的方式，对受保护系统的性能影响很小或几乎没有
- 网络监视器对网络中的用户是透明的，降低了本身遭受攻击的可能性
- 网络监听也可以发现对主机的基于网络协议的攻击方法（比如畸形网络包）
- 与受保护的主机操作系统无关



入侵监测相关的数学模型



- (1) 试验模型
- (2) 平均值和标准差模型
- (3) 多变量模型
- (4) 马尔可夫过程模型
- (5) 时序模型



CIDF规格文档

CIDF的规格文档由四部分组成，分别为：

- 体系结构：阐述了一个标准的**IDS**的通用模型
- 规范语言：定义了一个用来描述各种检测信息的标准语言
- 内部通讯：定义了**IDS**组件之间进行通信的标准协议
- 程序接口：提供了一整套标准的应用程序接口



CIDF的体系结构



- **CIDF**定义了**IDS**系统和应急系统之间通过交换数据方式，协作而实现入侵检测和应急响应。**CIDF**互操作有下面三类：
 - (1) 配置互操作，可相互发现并交换数据；
 - (2) 语法互操作，可正确识别交换的数据；
 - (3) 语义互操作，可相互正确理解交换的数据



Components



- ❑ 事件产生器 (Event generators)
- ❑ 事件分析器 (Event analyzers)
- ❑ 事件数据库 (Event databases)
- ❑ 响应单元 (Response units)



通信层次

- **CIDF**将各组件之间的通信划分为三个层次结构：**GIDO**层（**GIDO layer**）、消息层（**Message layer**）和传输层（**Negotiated Transport layer**）
- 其中传输层不属于**CIDF**规范，它可以采用很多种现有的传输机制来实现
- 消息层负责对传输的信息进行加密认证，然后将其可靠地从源传输到目的地，消息层不关心传输的内容，它只负责建立一个可靠的传输通道
- **GIDO**层负责对传输信息的格式化，正是因为有了**GIDO**这种统一的信息表达格式，才使得各个**IDS**之间的互操作成为可能



编写Snort的规则

- **snort**的每条规则都可以分成逻辑上的两个部分：规则头和规则选项
- 规则头包括：规则行为(**rule's action**)、协议(**protocol**)、源/目的**IP**地址、子网掩码以及源/目的端口。
- 规则选项包含报警信息和异常包的信息(特征码,**signature**)，使用这些特征码来决定是否采取规则规定的行动。



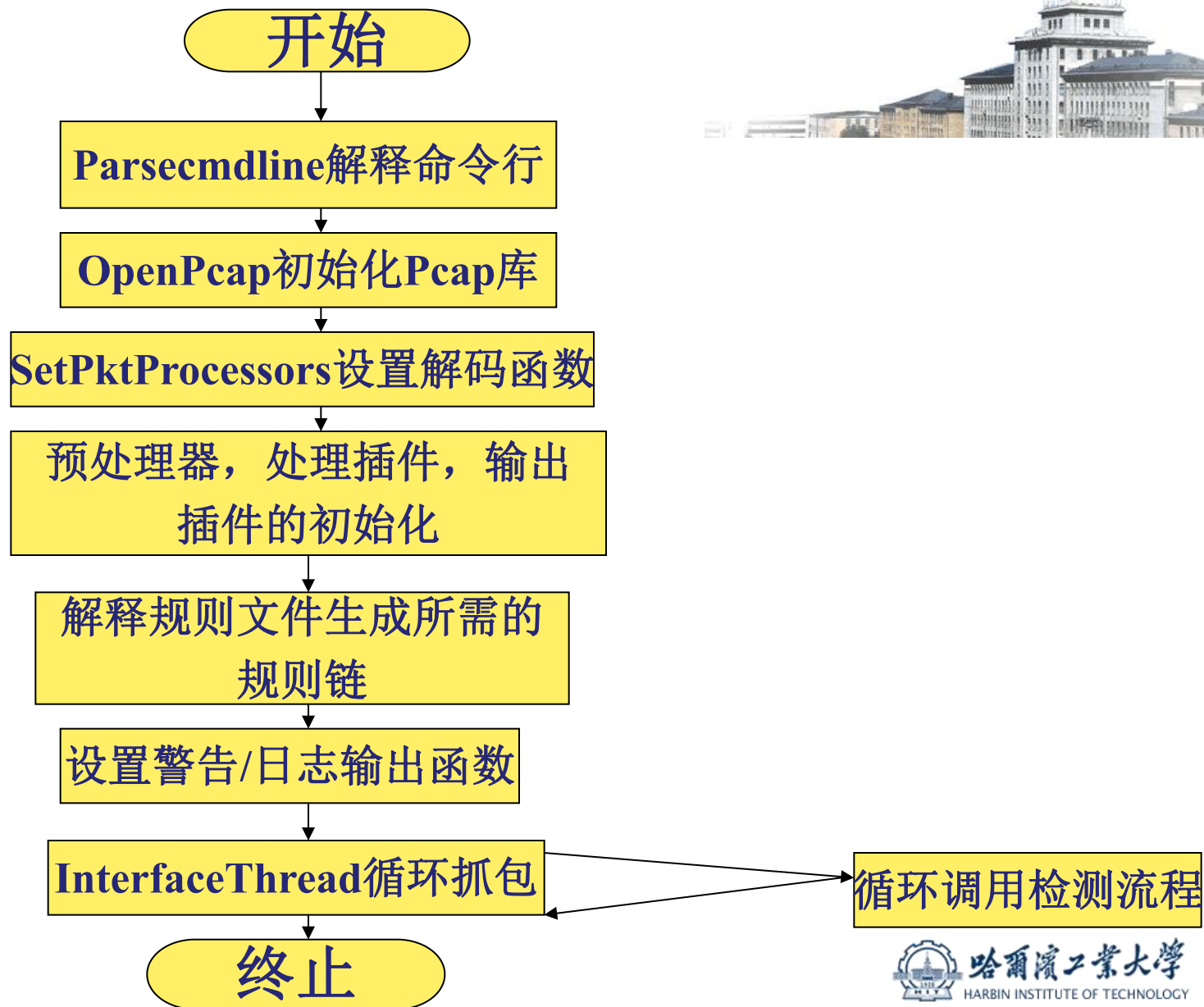
Snort的规则文件的语法



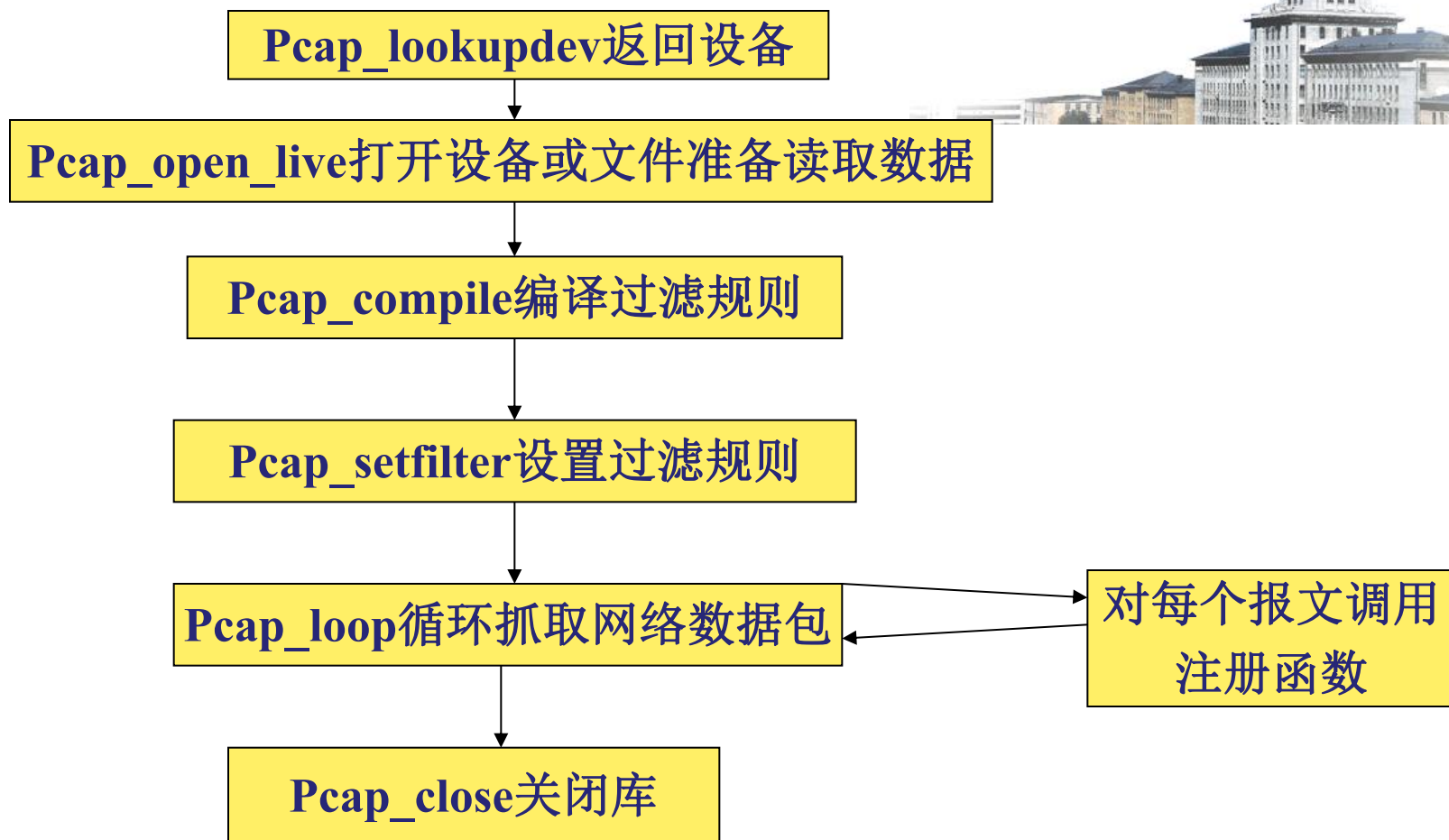
规则
变量定义
Include
预处理加载
输出加载



Snort总体流程



通用Libpcap应用程序流程



简单模式匹配技术的缺点



- 如果**IDS**采用的检测技术主要是简单的模式匹配，那么几乎总有办法避开检测。
- 一种攻击方法的本质特征不在于攻击代码中出现哪些特殊字符
- 简单模式匹配的办法在检测病毒是有效，但是网络攻击要比病毒复杂得多
- 在检测病毒时，只需考虑可执行文件的格式就可以了，但网络中存在着各种协议和**OS**，攻击者拥有灵活性



IDS的根本问题

- 为什么通过一些简单的变换，攻击者就可以逃避检测
 - 根本的原因是传统的**IDS**只能根据固定的规则和规定的匹配模式进行检测，**IDS**并不理解这些操作对于操作系统和应用程序意味着什么，**IDS**很难理解通过环境变量的组合构造的操作命令
- 通过更新规则的办法来反击隐秘攻击，**IDS**要付出多大的努力
 - 如果我们希望**IDS**能够理解各式各样的组合和变换，实际上我们就相当于要求**IDS**完成大部分本由操作系统完成的任务，这实际是不可能的



IDS在安全体系结构中层次的观点



- 检测应该处在与保护基本不交叉的一个独立的层次上，这种类型的**IDS**易实现
- 由于与**OS**无关所以可靠性较好，现在大多数基于网络的**IDS**基本上体现了这一观点
- 由于两个层次没有交叉，因此不能信息共享，应该有**OS**和应用程序完成的工作必须由**IDS**完成，比如协议还原，**Unicode**解码工作
- 加重了**IDS**的负担，同时由于**IDS**对协议的理解与**OS**和应用程序可能并不完全一致，也就隐含了误报和漏报的可能性



IDS在安全体系结构中层次的两点观点



- 检测应当与保护紧密结合，不是覆盖在系统之上的独立层次，而应该与系统的其他部分有机的结合在一起
- 在实际中通常**IDS**以内核补丁或驱动程序的形式出现。
- 这种方法往往能够以非常简洁的方法达到准确监测的目的。



响应类型



- 被动响应：只记录问题和报告问题
- 主动响应：用自动的或用户指定的方式，阻断攻击过程。



主动响应(active response)



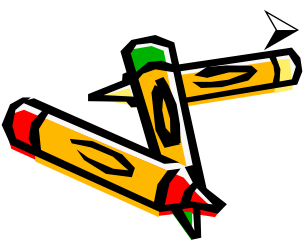
- 可分为三类：
 - 针对入侵者采取的措施
 - 修正系统
 - 收集更详细的信息



入侵响应小结



- 入侵响应应该与安全策略相协调。如在安全防护策略中规定一怎样的措施来对检测到的入侵行为作出响应。
- 按照措施的时间和紧急程度分为：
 - 即时措施：入侵发生时
 - 及时措施：入侵被完全检测出来时
 - 本地长期措施：处于本地安全的长期考虑
 - 全局长期措施：对社会安全状况很重要



基本概念



- 堡垒主机:Bastion Host
- 堡垒主机是一种配置了安全防范措施的网络上的计算机, 堡垒主机为网络之间的通信提供了一个阻塞点, 也就是说如果没有堡垒主机, 网络之间将不能相互访问。
- 双宿主机: Dual-homed Host
 - 有两个网络接口的计算机系统, 一个接口接内部网, 一个接口接外部网。
- DMZ(Demilitarized Zone, 非军事区或者停火区)
 - 在内部网络和外部网络之间增加的一个子网



防火墙的局限性(1)

- 1. 为了提高安全性，限制或关闭了一些有用但存在安全缺陷的网络服务，给用户带来使用的不便。
- 2. 目前防火墙对于来自网络内部的攻击还无能为力。
- 3. 防火墙不能防范不经过防火墙的攻击，如内部网用户通过SLIP 或PPP 直接进入Internet。
- 4. 防火墙对用户不完全透明，可能带来传输延迟、瓶颈及单点失效。



防火墙的局限性(2)



- 5.防火墙也不能完全防止受病毒感染的文件或软件的传输，由于病毒的种类繁多，如果要在防火墙完成对所有病毒代码的检查，防火墙的效率就会降到不能忍受的程度。
- 6.防火墙不能有效地防范数据驱动式攻击。
- 7.作为一种被动的防护手段，防火墙不能防范因特网上不断出现的新的威胁和攻击。



包过滤防火墙



- 往往用一台路由器来实现
- 基本的思想很简单
 - 对所接收的每个数据包进行检查，根据过滤规则，然后决定转发或者丢弃该包
 - 往往配置成双向的
- 如何过滤
 - 过滤规则基于**IP**包头信息。**IP**源地址、目的地址、**TCP/UDP**端口、**ICMP**消息类型、**TCP**头中的**ACK**位。
 - 过滤器往往建立一组规则，根据**IP**包是否匹配规则中指定的条件来作出决定。
 - 如果有匹配按规则执行，没有匹配，则按缺省策略。



包过滤操作的六项要求



- 六项要求:

① 包过滤设备必须存

② 当包到达端口时，
中的字段。

③ 包过滤规则的存储顺序与应用顺序相同；

④ 如果一条规则阻

⑤ 如果一条规则允
常通行；

⑥ 如果一个包不满足任何一条规则，该包就被
阻塞。

规则4、5说明规则以正
确顺序放置很重要

规则6依据的原理是：未明
确表示允许的便被禁止。



屏蔽主机模式特点

- 在这种方式的防火墙中，堡垒主机安装在内部网络上，通常在路由器上设立过滤规则，并使这个堡垒主机成为从外部网络唯一可直接到达的主机，这确保了内部网络不受未被授权的外部用户的攻击。屏蔽主机防火墙实现了网络层和应用层的安全，因而比单独的包过滤或应用网关代理更安全。在这一方式下，过滤路由器是否配置正确是这种防火墙安全与否的关键，如果路由表遭到破坏，堡垒主机就可能被越过，使内部网完全暴露。



屏蔽子网模式特点



- 网络管理员将堡垒主机，WEB服务器、Mail服务器等公用服务器放在非军事区网络中。内部网络和外部网络均可访问屏蔽子网，但禁止它们穿过屏蔽子网通信。在这一配置中，即使堡垒主机被入侵者控制，内部网仍受到内部包过滤路由器的保护。
- 堡垒主机运行各种代理服务，可以更有效地提供服务。



动态访问控制列表（动态包过滤技术）

配置动态访问控制列表，可以实现指定用户的**IP**数据流临时通过防火墙，进行会话连接，从而实现对数据包的动态过滤。当动态访问控制列表被触发后，动态访问控制列表重新配置接口上已有的访问控制列表，允许指定的用户访问指定的**IP**地址。在会话结束后，将接口配置恢复到原来的状态。

动态包过滤技术一般结合身份认证机制实现。例如，用户首先发起一个到防火墙的标准**Telnet**会话，防火墙进行身份验证。如果用户通过身份验证，则激活动态访问控制列表，在防火墙开放一个数据通道，此时，用户便可以暂时通过防火墙访问内部网络目标主机。



状态包检查(SPI, Stateful Packet Inspection)技术

又称为反射访问控制列表技术。**SPI**技术能够动态建立访问控制列表条目，在这些临时条目中不仅包含必要的包过滤信息，还包含了网络会话状态信息。这些临时条目在新会话开始（如内部主机向外部主机发起连接请求）时创建，并在会话结束时被删除。

SPI包过滤技术维护网络中的会话连接信息。防火墙检查每一个通过的数据包，看这些数据包是否属于一个已经建立的通过防火墙的会话的延续，或者能否通过预设定包过滤规则集的检查。**SPI**包过滤技术拒绝除设计允许的和已经在状态表中的连接之外的所有连接。



会话结束的判定和临时条目的删除

- 对于**TCP**会话，检测到两组**FIN**位被设置的**TCP**分组后，临时条目一般将在几秒钟内被删除，在检测到**RST**位被设置的**TCP**分组后，临时条目将被立即删除（在会话中的两组**FIN**位被置位的**TCP**分组表示会话即将结束，几秒钟的时间可以使得会话能完美地结束，设置了**RST**位的**TCP**分组表示会话突然关闭）。在超时时间段内，如果没有检测到和特定会话相关的数据分组，临时条目也将被删除。
- 对于**UDP**和其他协议，会话结束的判定方法不同于**TCP**。原因在于这些协议被认为是无连接服务，在它们的数据分组内没有会话跟踪信息，因此，通常在超时时间段内没有检测到此**UDP**会话的任何相关数据分组时，便认为**UDP**会话结束了。



NAT的作用



- 隐藏内部网络
- 解决**IP**地址不足
- 解决地址交迭
- 负载均衡



HTTP协议安全问题

- HTTP协议是一种简单的、无状态的应用层协议（RFC1945、RFC2616）
 - 无状态使攻击变得容易
 - 基于**ASCII**码，无需弄清复杂的二进制编码机制，攻击者就可了解协议中的明文信息
 - 互联网中存在的大量中间盒子，**HTTP**标准（RFC 2616和RFC 7320）的理解如果不一致，就有可能导致一些新的攻击发生（缓存投毒和过滤旁路）



Cookie安全问题



• Cookie的生成与维护

- 由服务器端生成，发送给客户端（一般是浏览器），浏览器会将**Cookie**的值保存到某个目录下的文本文件内，下次请求同一网站时就发送该**Cookie**给服务器（前提是浏览器设置为启用**Cookie**）
 - **Cookie**在生成时就会被指定一个**Expire**值，这就是**Cookie**的生存周期。到期自动清除。如果没有指定则表示浏览器关闭为止
 - 服务器可以利用**Cookie**存储信息并经常性地维护这些信息，从而判断在**HTTP**传输中的状态
 - 如果一台计算机上安装了多个浏览器，每个浏览器都会在各自独立的空間存放**Cookie**
 - 同一台计算机同一浏览器的不同用户的**Cookie**也是独立的
- Cookie**中的内容大多数经过了编码处理



Cookie安全问题

- **Cookie**中包含了一些敏感信息，如用户名、计算机名、使用的浏览器和曾经访问的网站等，攻击者可以利用它来进行窃密和欺骗攻击



注入漏洞



- 最普遍的注入漏洞包括：
 - **SQL**注入：通过**SQL**语句恶意地调用后台数据库
 - 系统调用
 - 通过**shell**命令调用外部程序

任何依赖于解释执行的Web应用都有被注入漏洞攻击的危险！



SQL注入原理



- 利用**web**应用程序对特殊字符串过滤不完全的缺陷
- 大多数的SQL注入攻击发生在**web**应用程序使用用户提供的输入内容来拼接动态**SQL**语句以访问数据库时



SQL注入原理



- 例子：
 - 通过用户提供的参数来查询表中的数据
 - "SELECT * FROM USERS WHERE SSN=" + **ssn** + ""
- SSN参数来自于用户的输入：
 - 参数未经验证或编码
 - 黑客输入： **1234' OR '1'='1**
- 应用程序构造查询语句：
 - SELECT * FROM USERS WHERE SSN=**'1234' OR '1'='1'**
 - 结果返回数据库中的每一个用户

永真逻辑
!



SQL注入：检测

- 在形如`http://xxx.xxx.xxx/abc.asp?id=XX`的带有参数的ASP动态网页中，XX为参数。参数的个数和类型取决于具体的应用。参数的类型可以是整型或者字符串型。下面我们以`http://xxx.xxx.xxx/abc.asp?id=YY`为例进行分析
 - 1、整型参数时的SQL注入漏洞探测
 - 2、字符串型参数时的SQL注入漏洞探测
 - 3、特殊情况的处理



http://xxx.xxx.xxx/abc.asp?id=YY

- 1、整型参数时的SQL注入漏洞探测
 - 通常abc.asp中的SQL大致为
 - Select * from 表名 where 字段=YY
 - 在URL链接中附加一个单引号，即
http://xxx.xxx.xxx/abc.asp?id=YY '，运行异常
 - 在URL链接中附加一个and 1=1，即
http://xxx.xxx.xxx/abc.asp?id=YY and 1=1运行正常
 - 在URL链接中附加一个and 1=2，即
http://xxx.xxx.xxx/abc.asp?id=YY and 1=2运行异常



http://xxx.xxx.xxx/abc.asp?id=YY

➤ 2、字符参数时的SQL注入漏洞探测

➤ 通常abc.asp中的SQL大致为

- Select * from 表名 where 字段= 'YY'

➤ 在URL链接中附加一个单引号，即

http://xxx.xxx.xxx/abc.asp?id=YY '，运行异常

➤ 在URL链接中附加一个 'and ' 1 '=' 1，即

http://xxx.xxx.xxx/abc.asp?id=YY 'and ' 1 '=' 1运行正常

➤ 在URL链接中附加一个 'and ' 1 '=' 2，即

http://xxx.xxx.xxx/abc.asp?id=YY 'and ' 1 '=' 2运行异常



SQL注入漏洞的防护

- 1. 字符串长度验证，仅接受指定长度范围内的变量值。
。sql注入脚本必然会大大增加输入变量的长度，通过长度限制，比如用户名长度为 8 到 20 个字符之间，超过就判定为无效值。
- 2. 对单引号和双“-”、下划线、百分号等sql注释符号进行转义
- 3. 对接收的参数进行类型格式化，如id参数值获取后，进行int类型转换



SQL注入漏洞的防护

- 4. 永远不要使用动态拼装SQL，推荐使用参数化的SQL或者直接使用存储过程进行数据查询存取。**sql**注入最主要的攻击对象就是动态拼装的SQL，通过参数化查询可以极大减少SQL注入的风险。
- 5. 永远不要使用管理员权限的数据库连接(sa、root、admin)，为每个应用使用单独的专用的低特权账户进行有限的数据库连接。
- 6. 不要把机密信息明文存放，请加密或者hash掉密码和敏感的信息。这样对方就算获取到整个表的数据内容，也没什么价值。



SQL注入漏洞的防护



- 7. 应用的异常信息应该给出尽可能少的提示，最好使用自定义的错误信息对原始错误信息进行包装，把异常信息输出到日志而不是在页面中展示。
- 8. 做好XSS跨站攻击的防护，防止攻击者伪造管理员信息进入系统后台
- 9. 不管客户端是否做过数据校验，在服务端必须要有数据校验（长度、格式、是否必填等等）



跨站脚本攻击



- 跨站脚本攻击（**XSS**）是指攻击者利用**Web**程序对用户输入过滤不足的缺陷，把恶意代码注入其他的用户浏览器显示的页面上执行，从而窃取用户敏感信息、伪造用户身份进行恶意行为的一种攻击方式





- **Xss**攻击的前提

- **Web**服务必须接受用户的输入，不仅包括url中的参数和表单字段，还包括**HTTP**头部和**cookie**值
- **Web**程序必须**重新显示**用户输入的内容，只有用户浏览器将**web**程序的数据解释成为**html**标记时，攻击才会发生。



跨站脚本攻击三种形式



- 反射式跨站脚本攻击
- 存储式跨站脚本攻击
- **DOM**式跨站脚本攻击



储存式XSS



- **Bob**拥有一个**Web**站点，该站点允许用户发布信息/浏览已发布的信息。
- **Charly**注意到**Bob**的站点具有类型**C**的**XSS**漏洞。
- **Charly**发布一个热点信息，吸引其它用户纷纷阅读。
- **Bob**或者是任何的其他人如**Alice**浏览该信息，其会话**cookies**或者其它信息将被**Charly**盗走。



反射式XSS



- (1) **Alice**经常浏览**Bob**建立的网站。**Bob**的站点运行**Alice**使用用户名/密码进行登录，并存储敏感信息(比如银行帐户信息)；
- (2) **Charly**发现**Bob**的站点包含反射性的XSS漏洞；
- (3) **Charly**编写一个利用漏洞的URL，并将其冒充为来自**Bob**的邮件发送给**Alice**；
- (4) **Alice**在登录到**Bob**的站点后，浏览**Charly**提供的URL；
- (5) 嵌入到URL中的恶意脚本在**Alice**的浏览器中执行，就像它直接来自**Bob**的服务器一样。此脚本盗窃敏感信息(授权、信用卡、帐号信息等)，然后在**Alice**完全不知情的情况下将这些信息发送到**Charly**的Web站点。



反射式XSS



- 反射型XSS的攻击步骤如下：
 1. 攻击者在url后面的参数中加入恶意攻击代码
 2. 当用户打开带有恶意代码的URL的时候，网站服务端将恶意代码从URL中取出，拼接在html中并且返回给浏览器端。
 3. 用户浏览器接收到响应后执行解析，其中的恶意代码也会被执行到。
 4. 攻击者通过恶意代码来窃取到用户数据并发送到攻击者的网站。攻击者会获取到比如cookie等信息，然后使用该信息来冒充合法用户的行为，调用目标网站接口执行攻击等操作。
- 常见的反射性XSS有哪些？
- 常见的是：恶意链接。



DOM式XSS



- dom就是通过JavaScript可以重构整个HTML文档，可以添加、移除、改变或者重排页面上的项目。
- 要改变页面上的某个东西，JavaScript就需要获得对html文档中所有元素进行访问的入口，这个入口，连同对html元素进行添加、移动、改变或移除的方法和属性，都是通过文档对象模型来获得的。

